

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Acosta Solis Hanna Aide Freire Arebalo Alan Anderson Landeta Tapia Edison Paul Quevedo Ajon Gina Anahi Rivadeneira Ulloa Matias Sebastián
Asignatura:	Algoritmos y Lógica de la Programación
Docente:	Ing. José Ruben Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

.1 Objetivos

General:

Manipular archivos mediante métodos y clases que proporciona el lenguaje de programación JAVA.

Específicos:

- Crear clases y métodos para pedir datos por teclado, hacer cálculos como el promedio y controlar el comportamiento del programa.
- Guardar y añadir información en un archivo externo sin borrar lo que ya estaba escrito antes, usando herramientas de almacenamiento en disco (FileWriter / ofstream).
- Hacer diagramas de flujo y pseudocódigo en PSeInt para revisar que la lógica del sistema funcione bien antes de pasarla a código real y tomar las capturas de pantalla como respectiva evidencia.

.2 Modalidad

Presencial

.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

.5 Listado de equipos, materiales y recursos

- Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel
- Material de escritorio

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

.6 Actividades por desarrollar

De acuerdo con la guía práctica SW-AyLP-APE-05: Creación, Lectura y Escritura de Ficheros, los ejercicios se enfocan en la manipulación de archivos utilizando clases y métodos del lenguaje Java y C++

.7 Resultados obtenidos

Ejercicio 1 – Registro de Estudiantes

1. Análisis

El ejercicio consiste en gestionar el registro de estudiantes capturando sus datos personales y académicos, específicamente cédula, nombre, carrera y promedio, para almacenarlos de manera persistente en un archivo de texto. Para lograrlo, se implementa una clase que encapsula estos atributos y define métodos encargados tanto de la entrada de datos por teclado como de la escritura en el archivo bajo un modo de anexo, garantizando así que la información se conserve correctamente. El proceso culmina confirmando al usuario que la operación fue exitosa y cerrando el flujo del archivo para asegurar la integridad de los datos registrados.

2. Declaración de Variables

Variable	Tipo de Dato	Propósito
cedula	string	Almacenar el número de identificación
nombre	string	Almacenar el nombre completo
carrera	string	Almacenar el nombre de la carrera
promedio	float	Almacenar el promedio académico

3. Algoritmo

Inicio

1. Definir variables (cédula, nombre, carrera, promedio).
2. Solicitar y leer los datos del estudiante.
3. Abrir el archivo "estudiantes.txt" en modo anexo.
4. Escribir los datos en el archivo.

5. Cerrar el archivo.
6. Mostrar mensaje de confirmación.

Fin

4. Seudocódigo

Algoritmo RegistroEstudiantes

Definir cedula, nombre, carrera Como Caracter

Definir promedio Como Real

Definir archivo Como Entero

// Solicitud de datos

Escribir "Ingrese cedula:"

Leer cedula

Escribir "Ingrese nombre:"

Leer nombre

Escribir "Ingrese carrera:"

Leer carrera

Escribir "Ingrese promedio:"

Leer promedio

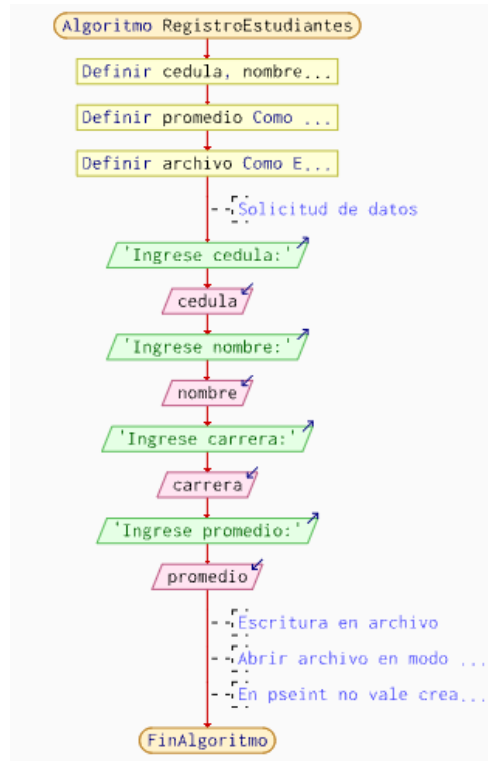
// Escritura en archivo

// Abrir archivo en modo añadir (append)

// En pseint no vale crear archivo txt. pero se ha diseñado una simulacion

FinAlgoritmo

5. Diagrama de Flujo



6. Código en C++

```

#include <iostream>

#include <fstream>

#include <string>

using namespace std;

class Estudiante {

private:

    string cedula, nombre, carrera;

    float promedio;

public:

    void capturarDatos() {

        cout << "Ingreso cedula: "; cin >> cedula;

        cout << "Ingreso nombre: "; cin >> nombre;

        cout << "Ingreso carrera: "; cin >> carrera;

        cout << "Ingreso promedio: "; cin >> promedio;

    }

}

```

```

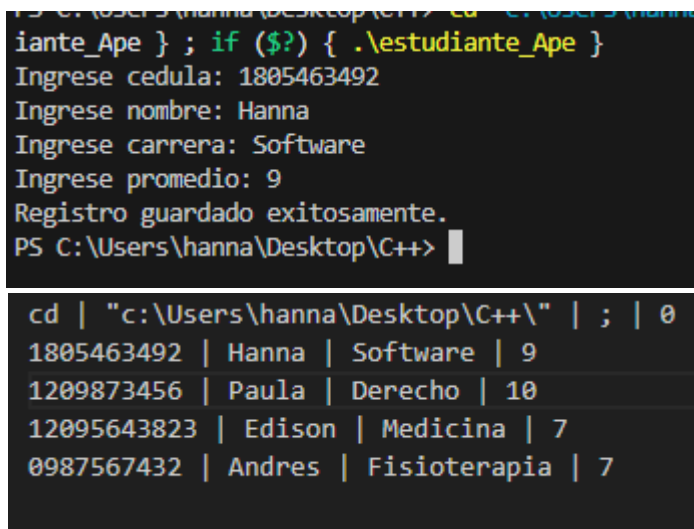
void guardarEnArchivo() {
    ofstream archivo("estudiantes.txt", ios::app);
    if (archivo.is_open()) {
        archivo << cedula << " | " << nombre << " | " << carrera << " | " <<
promedio << endl;
        archivo.close();
        cout << "Registro guardado exitosamente." << endl;
    } else {
        cout << "Error al abrir el archivo." << endl;
    }
}

};

int main() {
    Estudiante e;
    e.capturarDatos();
    e.guardarEnArchivo();
    return 0;
}

```

7. Prueba de escritorio C++



The screenshot shows a Windows command prompt window. The first part shows the execution of a C++ program where the user enters the following data: cedula: 1805463492, nombre: Hanna, carrera: Software, and promedio: 9. The program outputs "Registro guardado exitosamente." and returns to the prompt. The second part shows the user navigating to the directory "c:\Users\hanna\Desktop\C++\" and displaying the contents of the "estudiantes.txt" file. The file contains four lines of data, each separated by a vertical bar: "1805463492 | Hanna | Software | 9", "1209873456 | Paula | Derecho | 10", "12095643823 | Edison | Medicina | 7", and "0987567432 | Andres | Fisioterapia | 7".

```

PS C:\Users\hanna\Desktop\C++\> cd .\estudiante_Ape } ; if ($?) { .\estudiante_Ape }
Ingrese cedula: 1805463492
Ingrese nombre: Hanna
Ingrese carrera: Software
Ingrese promedio: 9
Registro guardado exitosamente.
PS C:\Users\hanna\Desktop\C++>

cd | "c:\Users\hanna\Desktop\C++\" | ; | 0
1805463492 | Hanna | Software | 9
1209873456 | Paula | Derecho | 10
12095643823 | Edison | Medicina | 7
0987567432 | Andres | Fisioterapia | 7

```

8. Código en Java

```

import java.io.FileWriter;
import java.io.PrintWriter;

```

```

import java.io.IOException;
import java.util.Scanner;

class Estudiante {

    private String cedula, nombre, carrera;

    private double promedio;

    public void capturarDatos() {

        Scanner sc = new Scanner(System.in);

        System.out.print("Ingrese cedula: "); cedula = sc.nextLine();

        System.out.print("Ingrese nombre: "); nombre = sc.nextLine();

        System.out.print("Ingrese carrera: "); carrera = sc.nextLine();

        System.out.print("Ingrese promedio: "); promedio = sc.nextDouble();

    }

    public void guardarEnArchivo() {

        try (PrintWriter writer = new PrintWriter(new FileWriter("estudiantes.txt",
true))) {

            writer.println(cedula + " | " + nombre + " | " + carrera + " | " +
promedio);

            System.out.println("Registro guardado exitosamente.");

        } catch (IOException e) {

            System.out.println("Error al escribir en el archivo: " + e.getMessage());

        }

    }

}

public class Main {

    public static void main(String[] args) {

        Estudiante e = new Estudiante();

        e.capturarDatos();

        e.guardarEnArchivo();

    }

}

```

9. Prueba de Escritorio Java

```

Ingreso cedula: 0984573218
Ingreso nombre: Edison
Ingreso carrera: Medicina
Ingreso promedio: 6
Registro guardado exitosamente.
PS C:\Users\hanna\Desktop\JAVA\EJERCICIO1_
NTES\" ; if ($?) { javac Mainm.java } ; if
Ingreso cedula: 1864736289
Ingreso nombre: Gina
Ingreso carrera: Enfermeria
Ingreso promedio: 10
Registro guardado exitosamente.

```

```

EJERCICIO1_ESTUDIANTES > estudiantes.txt
1 cd "c:\Users\hanna\Desktop\JAVA\EJERCICIO1_
2 1805463492 | Hanna | Software | 9.0
3 0984573218 | Edison | Medicina | 6.0
4 1864736289 | Gina | Enfermeria | 10.0
5 1803285438 | Andres | Derecho | 8.0
6

```

Ejercicio 2 – Lectura de Archivo de Estudiantes

1. Análisis

El programa lee un archivo de texto (estudiantes.txt) que contiene información de estudiantes en formato CSV con separador |. Cada línea representa un estudiante con: cédula, nombre, carrera y promedio. Luego:

- Lee y parsea cada línea del archivo
- Almacena los datos en un vector de estructuras Estudiante
- Muestra el listado completo en consola
- Calcula y muestra el promedio general de todos los estudiantes

2. Declaración de variables

Entrada

Variable	Tipo	Propósito
ruta	string	Nombre/ruta del archivo a leer
linea	string	Línea leída desde el archivo
pro	string	Promedio en formato texto antes de convertir
f	ifstream	Flujo de entrada del archivo

Proceso

Variable	Tipo	Propósito
lista	vector<Estudiante>	Almacena todos los estudiantes procesados

e	Estudiante	Estructura temporal para cada registro
ss	stringstream	Convierte línea en flujo para parsear
suma	double	Acumula la suma de los promedios
i	int	Índice para recorrer el vector

Salida

Variable	Tipo	Propósito
lista.size()	size_t	Total de estudiantes (se muestra)
suma / lista.size()	double	Promedio general calculado

3. Algoritmo

INICIO

1. Declarar variables:

- lista: vector de estudiantes
- linea, pro: cadenas
- suma: real
- f: archivo
- e: estructura estudiante

2. Abrir el archivo "estudiantes.txt" en modo lectura

3. MIENTRAS haya líneas por leer EN EL ARCHIVO HACER

3.1. Leer una línea del archivo y guardarla en 'linea'

3.2. SI la línea NO está vacía ENTONCES

3.2.1. Crear un stringstream con la línea

3.2.2. Extraer cédula (hasta encontrar '|')

3.2.3. Extraer nombre (hasta encontrar '|')

3.2.4. Extraer carrera (hasta encontrar '|')

3.2.5. Extraer promedio como cadena (hasta encontrar '|')

3.2.6. Convertir la cadena de promedio a número real

3.2.7. Guardar el estudiante en el vector 'lista'

FIN SI

FIN MIENTRAS

4. Cerrar el archivo

5. SI el vector 'lista' está vacío ENTONCES

5.1. Mostrar mensaje "No hay registros en el archivo"

5.2. Terminar el programa

FIN SI

6. Mostrar encabezado "=== Listado de Estudiantes ==="

7. suma = 0

8. PARA CADA estudiante 'e' EN 'lista' HACER

8.1. Mostrar: cédula, nombre, carrera y promedio del estudiante

8.2. suma = suma + e.promedio

FIN PARA

9. Mostrar "Total de estudiantes: " y la cantidad de elementos en 'lista'

10. Calcular promedio_general = suma / cantidad de estudiantes

11. Mostrar "Promedio general: " y promedio_general

FIN

4. Pseudo código en pseint

Algoritmo GestionEstudiantes

Definir lista Como Cadena

Dimensionar lista[100] // Arreglo de hasta 100 estudiantes

Definir e_cedula, e_nombre, e_carrera, e_promedio_txt Como Cadena

Definir e_promedio, suma, promedio_general Como Real

Definir i, total Como Entero

Definir archivo Como Cadena

// Entrada: nombre del archivo

archivo <- "estudiantes.txt"

// Proceso: leer archivo y llenar arreglo

total <- LeerArchivo(archivo, lista, e_cedula, e_nombre, e_carrera, e_promedio)

// Salida: mostrar resultados

```

Si total = 0 Entonces
    Escribir "No hay registros en el archivo."
Sino
    Escribir "=== Listado de Estudiantes ==="
    suma <- 0
    Para i <- 1 Hasta total Hacer
        // Obtener datos de la estructura (simulado con arreglos paralelos)
        e_cedula <- lista[i,1]
        e_nombre <- lista[i,2]
        e_carrera <- lista[i,3]
        e_promedio <- ConvertirANumero(lista[i,4])

        Escribir "Cedula: ", e_cedula, " | Nombre: ", e_nombre, " | Carrera: ",
e_carrera, " | Promedio: ", e_promedio

        suma <- suma + e_promedio

    FinPara
    Escribir ""
    Escribir "Total de estudiantes : ", total
    promedio_general <- suma / total
    Escribir "Promedio general      : ", promedio_general
FinSi
FinAlgoritmo

```

```

SubProceso total <- LeerArchivo(ruta, Por Referencia lista, Por Referencia ced, Por
Referencia nom, Por Referencia car, Por Referencia prom)

```

```

    Definir archivo Como Cadena
    Definir linea, partes, campo Como Cadena
    Definir i, pos, total Como Entero
    Definir encontrado Como Logico
    total <- 0
    archivo <- ruta
    i <- 1

    // Abrir archivo (simulación en PSeInt)

```

```

Escribir "Leyendo archivo: ", archivo

// Simulación de datos precargados (en PSeInt no hay lectura directa de archivos)
// Equivalentemente, asignamos los datos del ejemplo:
// NOTA: En PSeInt real se usa "Leer" desde teclado o archivo plano
// Para este ejercicio, simulamos el contenido del archivo:
// Simulación de líneas del archivo estudiantes.txt

Dimensionar lineas[3]

lineas[1] <- "12345678|Juan Perez|Ingenieria|8.5"
lineas[2] <- "87654321|Maria Gomez|Medicina|9.2"
lineas[3] <- "11223344|Carlos Ruiz|Derecho|7.8"

Para i <- 1 Hasta 3 Hacer
    linea <- lineas[i]
    Si Longitud(linea) > 0 Entonces
        total <- total + 1
        // Parsear por '|'
        pos <- 1
        // Extraer cédula
        campo <- ""
        Mientras Subcadena(linea, pos, pos) <> "|" Y pos <= Longitud(linea) Hacer
            campo <- campo + Subcadena(linea, pos, pos)
            pos <- pos + 1
        FinMientras
        lista[total,1] <- campo
        pos <- pos + 1 // saltar '|'

        // Extraer nombre
        campo <- ""
        Mientras Subcadena(linea, pos, pos) <> "|" Y pos <= Longitud(linea) Hacer
            campo <- campo + Subcadena(linea, pos, pos)
            pos <- pos + 1
        FinMientras

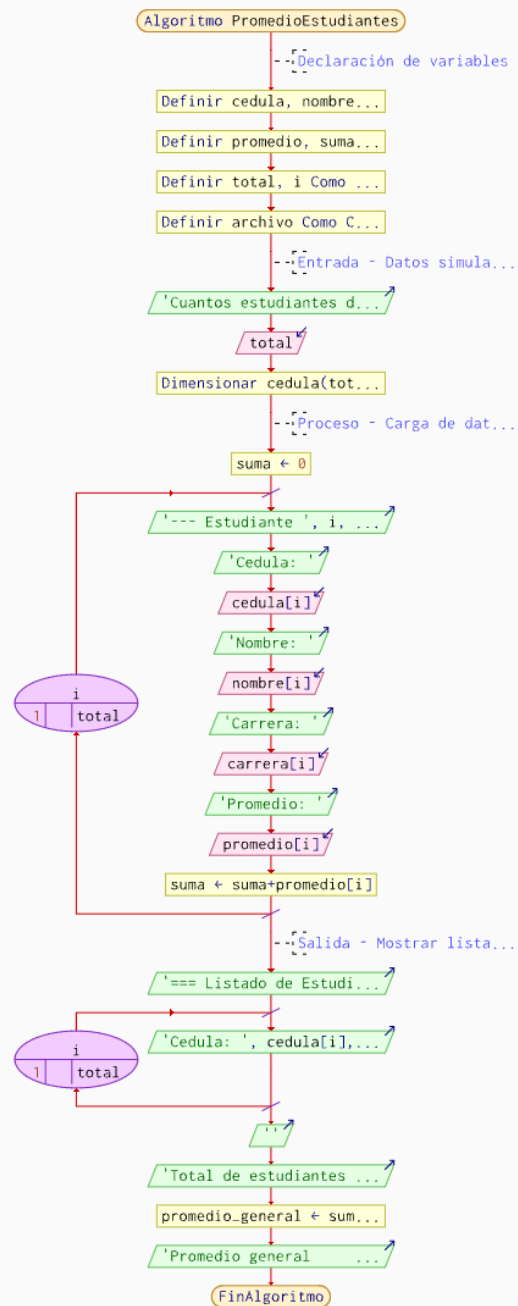
```

```

        lista[total,2] <- campo
        pos <- pos + 1
        // Extraer carrera
        campo <- ""
        Mientras Subcadena(linea, pos, pos) <> "|" Y pos <= Longitud(linea) Hacer
            campo <- campo + Subcadena(linea, pos, pos)
            pos <- pos + 1
        FinMientras
        lista[total,3] <- campo
        pos <- pos + 1
        // Extraer promedio
        campo <- ""
        Mientras pos <= Longitud(linea) Hacer
            campo <- campo + Subcadena(linea, pos, pos)
            pos <- pos + 1
        FinMientras
        lista[total,4] <- campo
    FinSi
FinPara
FinSubProceso

```

5. Diagrama de flujo



6. Código C++

```

#include <iostream>

#include <fstream>

#include <sstream>

#include <string>

#include <vector>

```

```

using namespace std;

struct Estudiante { string cedula, nombre, carrera; double promedio; };

// Lee y parsea cada línea del archivo
vector<Estudiante> leerArchivo(const string& ruta) {
    vector<Estudiante> lista;
    ifstream f(ruta);
    string linea;
    while (getline(f, linea)) {
        if (linea.empty()) continue;
        stringstream ss(linea);
        Estudiante e;
        string pro;
        getline(ss, e.cedula, '|');
        getline(ss, e.nombre, '|');
        getline(ss, e.carrera, '|');
        getline(ss, pro, '|');
        e.promedio = stod(pro);
        lista.push_back(e);
    }
    return lista;
}

int main() {
    auto lista = leerArchivo("estudiantes.txt");
    if (lista.empty()) {
        cout << "No hay registros en el archivo." << endl;
        return 0;
    }
    cout << "=== Listado de Estudiantes ===" << endl;
    double suma = 0;

```

```

for (auto& e : lista) {

    cout << "Cedula: " << e.cedula << " | Nombre: " << e.nombre

        << " | Carrera: " << e.carrera

        << " | Promedio: " << e.promedio << endl;

    suma += e.promedio;

}

cout << "\nTotal de estudiantes : " << lista.size() << endl;

cout << "Promedio general      : " << suma / lista.size() << endl;

return 0;

}

```

7. Prueba de escritorios C++

```

=== Listado de Estudiantes ===
Cedula: 1650011404 | Nombre: Skarlett | Carrera: Contabilidad | Promedio: 0
Cedula: 1751094523 | Nombre: ALAN | Carrera: SOFTWARE | Promedio: 10
Cedula: 1677777778 | Nombre: Fernando | Carrera: TICS | Promedio: 9
Cedula: 1234231232 | Nombre: Patricia | Carrera: Medicina | Promedio: 7
Cedula: 1650143934 | Nombre: Julieta | Carrera: Marketing | Promedio: 0

Total de estudiantes : 5
Promedio general    : 5.2

```

8. Código En Java

```

import java.io.BufferedReader;

import java.io.FileReader;

import java.io.IOException;

import java.util.ArrayList;

import java.util.List;

class Estudiante {

    String cedula;

    String nombre;

    String carrera;

    double promedio;

    Estudiante(String cedula, String nombre, String carrera, double promedio) {

        this.cedula = cedula;

        this.nombre = nombre;

        this.carrera = carrera;

        this.promedio = promedio;
    }
}

```

```

    }
}

public class LecturaEstudiantes {

    public static List<Estudiante> leerArchivo(String ruta) {

        List<Estudiante> lista = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {

            String linea;

            while ((linea = br.readLine()) != null) {

                if (linea.trim().isEmpty()) continue;

                String[] partes = linea.split("\\|");

                if (partes.length == 4) {

                    String cedula = partes[0];

                    String nombre = partes[1];

                    String carrera = partes[2];

                    double promedio = Double.parseDouble(partes[3]);

                    lista.add(new Estudiante(cedula, nombre, carrera, promedio));

                }

            }

        } catch (IOException e) {

            System.err.println("Error al leer el archivo: " + e.getMessage());

        }

        return lista;

    }

    public static void main(String[] args) {

        List<Estudiante> lista = leerArchivo("estudiantes.txt");

        if (lista.isEmpty()) {

            System.out.println("No hay registros en el archivo.");

            return;

        }

        System.out.println("=== Listado de Estudiantes ===");

        double suma = 0;

        for (Estudiante e : lista) {

```



```

        System.out.printf("Cedula: %s | Nombre: %s | Carrera: %s | Promedio: %.2f%n",
                           e.cedula, e.nombre, e.carrera, e.promedio);

        suma += e.promedio;
    }

    System.out.println("\nTotal de estudiantes : " + lista.size());

    System.out.printf("Promedio general      : %.2f%n", suma / lista.size());

}
}

```

9. Prueba de escritorio java

```

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
=== Listado de Estudiantes ===
Cedula: 1650011404 | Nombre: Skarlett | Carrera: Contabilidad | Promedio: 0,00
Cedula: 1751094523 | Nombre: ALAN | Carrera: SOFTWARE | Promedio: 10,00
Cedula: 1677777778 | Nombre: Fernando | Carrera: TICS | Promedio: 9,00
Cedula: 1234231232 | Nombre: Patricia | Carrera: Medicina | Promedio: 7,00
Cedula: 1650143934 | Nombre: Julieta | Carrera: Marketing | Promedio: 0,00

Total de estudiantes : 5
Promedio general      : 5,20

```

Ejercicio 3 – Sistema de Inventario Básico

1. Análisis

El programa implementa un sistema básico de gestión de inventario que permite registrar productos y persistir los datos en un archivo de texto "inventario.txt". Cada producto cuenta con cuatro atributos: código, nombre, cantidad disponible y precio unitario. El sistema ofrece un menú interactivo con tres funcionalidades principales:

- Registrar producto: captura código, nombre, cantidad y precio, valida que cantidad y precio sean valores positivos, agrega el producto a la colección en memoria y lo guarda inmediatamente en el archivo en modo append.
- Mostrar inventario: lista todos los productos numerados, mostrando además el subtotal por producto (precio × cantidad).
- Calcular valor total: recorre todo el inventario sumando los subtotales para obtener el valor monetario total del inventario.

Al iniciar el programa, se cargan los productos previamente guardados desde el archivo. El diseño aplica Programación Orientada a Objetos mediante la clase Producto, que encapsula los atributos y el cálculo del subtotal, complementada con métodos separados para cada operación del sistema.

2. Declaración de Variables

Variable	Tipo	Descripción
NOMBRE_ARCHIVO	String (const)	Nombre del archivo de persistencia "inventario.txt"
codigo	String	Código único identificador del producto
nombre	String	Nombre descriptivo del producto
cantidad	int / Entero	Número de unidades del producto en stock
precio	double / Real	Precio unitario del producto en dólares
subtotal	double / Real	Resultado de $\text{precio} \times \text{cantidad}$ para un producto
total	double / Real	Suma acumulada de todos los subtotales del inventario
opcion	int / Entero	Opción seleccionada en el menú principal
inventario	vector / List	Colección de todos los objetos Producto cargados
linea	String	Variable temporal para leer cada línea del archivo

3. Algoritmo

1. Cargar el inventario existente desde "inventario.txt" hacia la colección en memoria.
2. Mostrar el menú principal con las opciones 1 (Registrar), 2 (Mostrar), 3 (Calcular total) y 0 (Salir).
3. Leer la opción ingresada por el usuario.
4. Si opción = 1: solicitar código, nombre, cantidad y precio; validar que cantidad y precio sean positivos; crear objeto Producto; añadir a la colección; guardar la línea en el archivo en modo append.
5. Si opción = 2: si el inventario está vacío mostrar mensaje; sino recorrer la colección y mostrar cada producto numerado con su subtotal.
6. Si opción = 3: inicializar total = 0; recorrer la colección sumando $\text{precio} \times \text{cantidad}$ de cada producto; mostrar el valor total final.
7. Si opción = 0: terminar el programa.
8. Para cualquier otra opción: mostrar mensaje de opción inválida.

9. Repetir desde el paso 2 hasta que la opción sea 0.

4. Pseudocódigo en PSeInt

```
Proceso SistemaInventario

    Definir codigo, nombre Como Cadena

    Definir cantidad, opcion Como Entero

    Definir precio, total Como Real

    CargarInventario("inventario.txt")

    Repetir

        Escribir "===== SISTEMA DE INVENTARIO ====="

        Escribir "1. Registrar producto"

        Escribir "2. Mostrar inventario"

        Escribir "3. Calcular valor total"

        Escribir "0. Salir"

        Leer opcion

    Segun opcion Hacer

        1:

            Escribir "Codigo  : ";   Leer codigo

            Escribir "Nombre  : ";   Leer nombre

            Escribir "Cantidad: ";   Leer cantidad

            Escribir "Precio  : ";   Leer precio

            Si cantidad >= 0 Y precio >= 0 Entonces

                AgregarProducto(codigo, nombre, cantidad, precio)
```

```

        GuardarEnArchivo("inventario.txt", codigo, nombre, cantidad,
precio)

        Escribir "Producto registrado correctamente"

SiNo

        Escribir "Error: cantidad y precio deben ser positivos"

FinSi

2:

Si inventario esta vacio Entonces

        Escribir "Inventario vacio"

SiNo

        Escribir "INVENTARIO COMPLETO"

Para i <- 0 Hasta largo(inventario) - 1 Hacer

        Escribir "Codigo  :", inventario[i].codigo

        Escribir "Nombre  :", inventario[i].nombre

        Escribir "Cantidad:", inventario[i].cantidad

        Escribir "Precio  :", inventario[i].precio

        Escribir "Subtotal: $", inventario[i].precio *
inventario[i].cantidad

        FinPara

FinSi

3:

total <- 0

Para i <- 0 Hasta largo(inventario) - 1 Hacer

        total <- total + inventario[i].precio * inventario[i].cantidad

FinPara

Escribir "Valor total del inventario: $", total

```

```

De Otro Modo:

    Escribir "Opcion invalida"

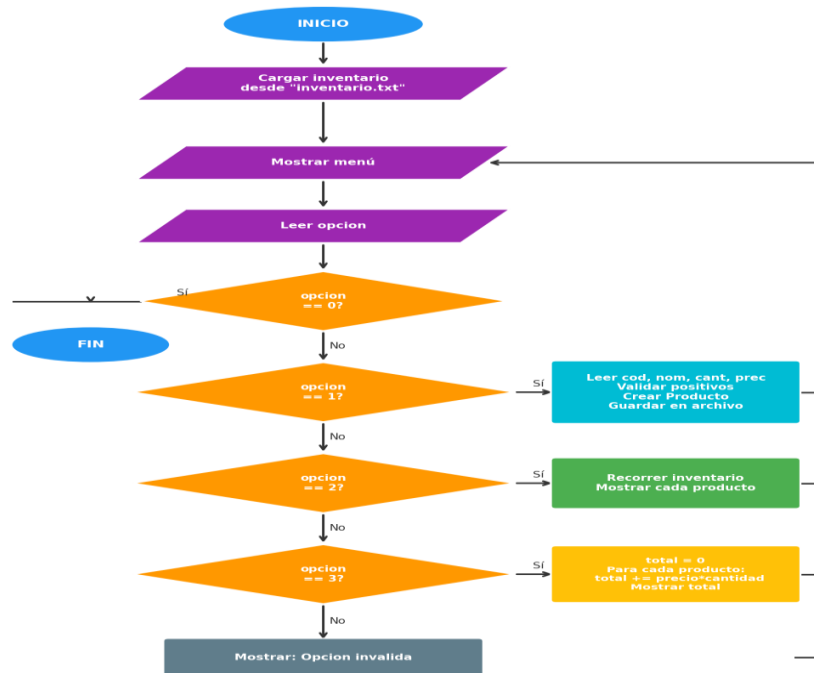
FinSegun

Hasta Que opcion = 0

FinProceso

```

5. Diagrama de Flujo



6. Código C++

```

// =====

// APE 05 - Ejercicio 3: Sistema de Inventario Basico

// Asignatura: Algoritmos y Logica de Programacion

```

```

// Docente: Jose Ruben Caiza

// =====

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>
#include <iomanip>
using namespace std;

class Producto {
public:
    string codigo, nombre;
    int    cantidad;
    double precio;

    Producto(string cod, string nom, int cant, double prec)
        : codigo(cod), nombre(nom), cantidad(cant), precio(prec) {}

    void mostrar() {
        cout << "  Codigo   : " << codigo << endl;
        cout << "  Nombre    : " << nombre << endl;
        cout << "  Cantidad: " << cantidad << endl;
        cout << "  Precio   : $" << fixed << setprecision(2) << precio << endl;
        cout << "  Subtotal: $" << fixed << setprecision(2) << (precio * cantidad)
        << endl;
    }
}

```

```

double subtotal() { return precio * cantidad; }

string toLinea() {
    stringstream ss; ss << fixed << setprecision(2) << precio;
    return codigo + "|" + nombre + "|" + to_string(cantidad) + "|" + ss.str();
}

};

vector<string> dividir(string linea, char delim) {
    vector<string> partes; stringstream ss(linea); string tok;
    while (getline(ss, tok, delim)) partes.push_back(tok);
    return partes;
}

void guardarProducto(string archivo, Producto& p) {
    ofstream f(archivo, ios::app);
    if (f.is_open()) { f << p.toLinea() << "\n"; f.close(); }
    else cout << "Error al abrir el archivo." << endl;
}

void registrarProducto(string archivo, vector<Producto>& inv) {
    string cod, nom; int cant; double prec;

    cout << "\n--- Registrar Nuevo Producto ---\nCodigo  : "; cin >> cod;
    cin.ignore();

    cout << "Nombre   : "; getline(cin, nom);

    cout << "Cantidad: "; cin >> cant;

    cout << "Precio   : "; cin >> prec;
}

```

```

        if (cant < 0 || prec < 0) { cout << "Error: valores negativos no permitidos."
<< endl; return; }

        Producto p(cod, nom, cant, prec);

        inv.push_back(p);

        guardarProducto(archivo, p);

        cout << "Producto registrado correctamente." << endl;

    }

```

```

vector<Producto> leerInventario(string archivo) {

    vector<Producto> inv; ifstream f(archivo);

    if (!f.is_open()) return inv;

    string linea;

    while (getline(f, linea)) {

        if (!linea.empty()) {

            auto d = dividir(linea, '|');

            if (d.size() == 4)

                inv.push_back(Producto(d[0], d[1], stoi(d[2]), stod(d[3])));

        }

    }

    f.close(); return inv;

}

```

```

void mostrarInventario(vector<Producto>& inv) {

    if (inv.empty()) { cout << "Inventario vacio." << endl; return; }

    cout << "\n===== \n";

    cout << "            INVENTARIO COMPLETO \n";

    cout << "  Total de productos: " << inv.size() << " \n";

    cout << "===== \n";

    for (int i = 0; i < (int)inv.size(); i++) {

```



```

        cout << "Producto #" << (i+1) << endl;

        inv[i].mostrar();

        cout << "-----\n";

    }

}

double calcularValorTotal(vector<Producto>& inv) {

    double total = 0.0;

    for (auto& p : inv) total += p.subtotal();

    return total;

}

int main() {

    string archivo = "inventario.txt";

    vector<Producto> inv = leerInventario(archivo);

    int op;

    do {

        cout << "\n===== SISTEMA DE INVENTARIO =====\n";

        cout << "1. Registrar producto\n2. Mostrar inventario\n";

        cout << "3. Calcular valor total\n0. Salir\nOpcion: ";

        cin >> op;

        switch (op) {

            case 1: registrarProducto(archivo, inv); break;

            case 2: mostrarInventario(inv); break;

            case 3: {

                double t = calcularValorTotal(inv);

                cout << "\nValor total del inventario: $"

                    << fixed << setprecision(2) << t << endl;

                break;

            }

        }

    } while (op != 0);

}

```

```

    }

    case 0: cout << "Saliendo..." << endl; break;

    default: cout << "Opcion invalida." << endl;

    }

    } while (op != 0);

    return 0;

}

```

7. Prueba de Escritorio – C++

```

===== SISTEMA DE INVENTARIO =====
1. Registrar producto
2. Mostrar inventario
3. Calcular valor total
0. Salir
Opcion: 1

--- Registrar Nuevo Producto ---
Codigo : INV001
Nombre : Teclado mecanico Redragon
Cantidad: 15
Precio : 45.00
Producto registrado correctamente en
inventario.txt.

Opcion: 1
Codigo : INV002
Nombre : Mouse Logitech G203
Cantidad: 8
Precio : 25.50
Producto registrado correctamente.

Opcion: 1
Codigo : INV003
Nombre : Monitor LG 24"
Cantidad: 5
Precio : 100.00
Producto registrado correctamente.

Opcion: 2

===== INVENTARIO COMPLETO =====
Total de productos: 3
=====
Producto #1
Codigo : INV001
Nombre : Teclado mecanico Redragon
Cantidad: 15
Precio : 45.00
Subtotal: 675.00
-----
Producto #2
Codigo : INV002
Nombre : Mouse Logitech G203
Cantidad: 8
Precio : 25.50
Subtotal: 204.00
-----
Producto #3
Codigo : INV003
Nombre : Monitor LG 24"
Cantidad: 5
Precio : 100.00
Subtotal: 500.00
-----

Opcion: 3
Valor total del inventario: $1779.00

Opcion: 0
Saliendo del sistema.

```

8.Código Java

```

// =====

// APE 05 - Ejercicio 3: Sistema de Inventario Basico

// Asignatura: Algoritmos y Logica de Programacion

// Docente: Jose Ruben Caiza

// =====

import java.io.*;

import java.util.*;

class Producto {

```

```
private String codigo, nombre;

private int    cantidad;

private double precio;


public Producto(String c, String n, int cant, double p) {

    codigo=c; nombre=n; cantidad=cant; precio=p;

}

public String getCodigo()    { return codigo;    }

public String getNombre()    { return nombre;    }

public int    getCantidad() { return cantidad; }

public double getPrecio()    { return precio;    }


public void mostrar() {

    System.out.println("  Codigo   : " + codigo);

    System.out.println("  Nombre   : " + nombre);

    System.out.println("  Cantidad: " + cantidad);

    System.out.printf ("  Precio   : $%.2f%n", precio);

    System.out.printf ("  Subtotal: $%.2f%n", subtotal());

}


public double subtotal() { return precio * cantidad; }


public String toLinea() {

    return codigo + "|" + nombre + "|" + cantidad + "|" +
String.format("%.2f", precio);

}

}
```

```

public class Ejercicio3 {

    static final String ARCHIVO = "inventario.txt";

    static List<Producto> inventario = new ArrayList<>();

    static Scanner sc = new Scanner(System.in);

    public static void guardarProducto(Producto p) {

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(ARCHIVO,
true))) {

            bw.write(p.toLinea()); bw.newLine();

        } catch (IOException e) { System.out.println("Error: " + e.getMessage());
}

    }

    public static void registrarProducto() {

        System.out.println("\n--- Registrar Nuevo Producto ---");

        System.out.print("Codigo   : "); String cod = sc.nextLine();

        System.out.print("Nombre   : "); String nom = sc.nextLine();

        System.out.print("Cantidad: "); int cant =
Integer.parseInt(sc.nextLine());

        System.out.print("Precio   : "); double prec =
Double.parseDouble(sc.nextLine());

        if (cant < 0 || prec < 0) { System.out.println("Error: valores
negativos."); return; }

        Producto p = new Producto(cod, nom, cant, prec);

        inventario.add(p);

        guardarProducto(p);

        System.out.println("Producto registrado correctamente.");

    }

    public static void leerInventario() {

        inventario.clear();
    }
}

```

```

        try (BufferedReader br = new BufferedReader(new FileReader(ARCHIVO))) {

            String l;

            while ((l = br.readLine()) != null) if (!l.isEmpty()) {

                String[] d = l.split("\\|");

                if (d.length == 4)

                    inventario.add(new Producto(d[0], d[1],
Integer.parseInt(d[2]),

                                Double.parseDouble(d[3])));

            }

        } catch (IOException e) { /* primer uso */ }

    }

    public static void mostrarInventario() {

        if (inventario.isEmpty()) { System.out.println("Inventario vacio.");
return; }

        System.out.println("\n=====");

        System.out.println("          INVENTARIO COMPLETO");

        System.out.println("  Total de productos: " + inventario.size());

        System.out.println("=====");

        for (int i = 0; i < inventario.size(); i++) {

            System.out.println("Producto #" + (i+1));

            inventario.get(i).mostrar();

            System.out.println("-----");

        }

    }

    public static double calcularValorTotal() {

        double total = 0.0;

        for (Producto p : inventario) total += p.subtotal();

    }

```

```

        return total;
    }

    public static void main(String[] args) {
        leerInventario();

        int op;

        do {

            System.out.println("\n===== SISTEMA DE INVENTARIO =====");

            System.out.println("1. Registrar producto\n2. Mostrar inventario");

            System.out.println("3. Calcular valor total\n0. Salir");

            System.out.print("Opcion: ");

            try { op = Integer.parseInt(sc.nextLine()); }

            catch (NumberFormatException e) { op = -1; }

            switch (op) {

                case 1: registrarProducto(); break;

                case 2: mostrarInventario(); break;

                case 3:

                    System.out.printf("%nValor total del inventario: $%.2f%n",
calcularValorTotal());

                    break;

                case 0: System.out.println("Saliendo..."); break;

                default: System.out.println("Opcion invalida.");

            }

        } while (op != 0);

        sc.close();

    }

}

```

9. Prueba de Escritorio – Java

```
===== SISTEMA DE INVENTARIO =====
1. Registrar producto
2. Mostrar inventario
3. Calcular valor total
0. Salir
Opcion: 1

--- Registrar Nuevo Producto ---
Codigo : INV001
Nombre : Teclado mecanico Redragon
Cantidad: 15
Precio : 45.00
Producto registrado correctamente en
inventario.txt.

Opcion: 1
Codigo : INV002
Nombre : Mouse Logitech G203
Cantidad: 8
Precio : 25.50
Producto registrado correctamente.

Opcion: 1
Codigo : INV003
Nombre : Monitor LG 24"
Cantidad: 5
Precio : 180.00
Producto registrado correctamente.

Opcion: 2

===== INVENTARIO COMPLETO =====
Total de productos: 3

-----
Producto #1
Codigo : INV001
Nombre : Teclado mecanico Redragon
Cantidad: 15
Precio : 45.00
Subtotal: 675.00

Producto #2
Codigo : INV002
Nombre : Mouse Logitech G203
Cantidad: 8
Precio : 25.50
Subtotal: 204.00

Producto #3
Codigo : INV003
Nombre : Monitor LG 24"
Cantidad: 5
Precio : 180.00
Subtotal: 900.00

-----

Opcion: 3
Valor total del inventario: 1779.00

Opcion: 0
Saliendo del sistema...
```

Ejercicio 4 – Agenda Telefónica

1. Analisis

Desarrollar un programa en C++ y Java que permita gestionar una agenda de contactos mediante archivos de texto. El sistema debe permitir agregar contactos con nombre, teléfono y correo, buscar un contacto por nombre y mostrar el listado completo. Los datos deben persistir en el archivo agenda.txt para que no se pierdan al cerrar el programa.

2. Declarar Variables

Entrada: nombre, teléfono, correo del contacto. Opción seleccionada en el menú. Nombre a buscar.

Proceso:

Clase Contacto con atributos privados: nombre, telefono, correo

Método mostrarContacto() para imprimir la información

Archivo agenda.txt abierto en modo append (ios::app / FileWriter(true)) para agregar sin borrar

Método toArchivo() para serializar el contacto con separador |

Método fromArchivo() para reconstruir el objeto desde una línea del archivo

stringstream (C++) / BufferedReader (Java) para leer línea a línea

Salida: Lista de contactos guardados. Resultado de búsqueda. Mensaje de confirmación o error.

3. Algoritmo

Inicio

Mostrar menú con opciones: Agregar / Mostrar / Buscar / Salir.

Leer opción del usuario.

Si la opción es **Agregar contacto:**

Pedir nombre, teléfono y correo.

Crear objeto Contacto con los datos ingresados.

Abrir agenda.txt en modo append.

Escribir: nombre|telefono|correo y salto de línea.

Cerrar el archivo y mostrar mensaje de confirmación.

Si la opción es **Mostrar contactos:**

Verificar si agenda.txt existe.

Leer línea por línea y reconstruir cada Contacto con fromArchivo().

Llamar a mostrarContacto() por cada registro.

Mostrar total de contactos al final.

Si la opción es **Buscar contacto**:

Pedir nombre a buscar.

Leer línea por línea del archivo.

Si el nombre contiene la cadena buscada, mostrar el contacto.

Si no se encuentra ninguno, mostrar mensaje de aviso.

Repetir hasta que el usuario elija Salir.

Fin

4. Pseudocodigo

```
Algoritmo AgendaTelefonica
// Límite máximo de almacenamiento en memoria para los contactos
Definir MAX_CONTACTOS Como Entero
MAX_CONTACTOS <- 100
// Arreglos paralelos
Definir nombre, telefono, correo Como Cadena
Dimensionar nombre(MAX_CONTACTOS)
Dimensionar telefono(MAX_CONTACTOS)
Dimensionar correo(MAX_CONTACTOS)
// Contador real de contactos almacenados
Definir totalContactos Como Entero
totalContactos <- 0
Definir opcion Como Entero
Escribir '===== '
Escribir '      Agenda Telefonica (Ej.4)      '
Escribir '===== '
Repetir
Escribir ''
Escribir ' MENU PRINCIPAL'
Escribir '  1. Agregar contacto'
Escribir '  2. Mostrar contactos'
Escribir '  3. Buscar contacto'
Escribir '  4. Salir'
Escribir ' Opcion: 'Sin Saltar
Leer opcion
Según opcion Hacer
1:
// --- AGREGAR CONTACTO ---
Si totalContactos<MAX_CONTACTOS Entonces
totalContactos <- totalContactos+1
Escribir ''
Escribir ' -- Agregar Contacto --'
Escribir ' Nombre : 'Sin Saltar
Leer nombre[totalContactos]
Escribir ' Telefono : 'Sin Saltar
Leer telefono[totalContactos]
Escribir ' Correo : 'Sin Saltar
Leer correo[totalContactos]
Escribir ' [OK] Contacto guardado correctamente.'
SiNo
Escribir ' [ERROR] Agenda llena. No se pueden agregar mas contactos.'
FinSi
2:
// --- MOSTRAR CONTACTOS ---
Definir i Como Entero
Si totalContactos=0 Entonces
```

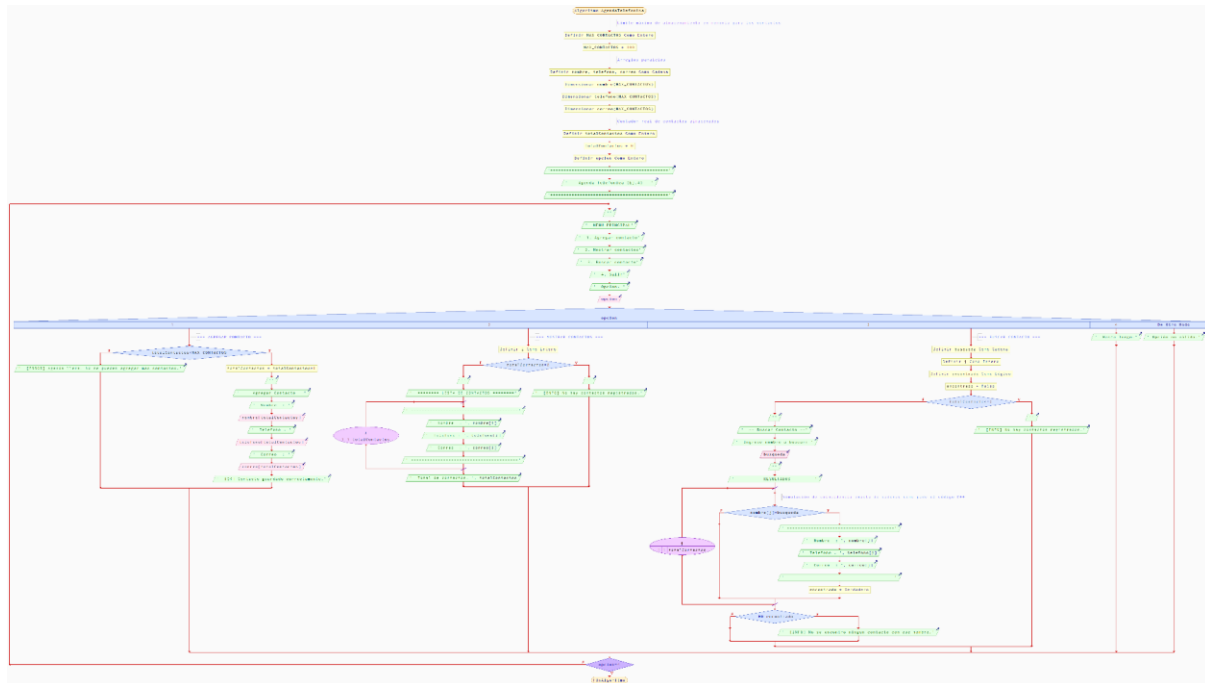


```

Escribir ''
Escribir ' [INFO] No hay contactos registrados.'
SiNo
Escribir ''
Escribir ' ===== LISTA DE CONTACTOS ====='
Para i<-1 Hasta totalContactos Con Paso 1 Hacer
Escribir ' -----'
Escribir ' Nombre : ', nombre[i]
Escribir ' Telefono : ', telefono[i]
Escribir ' Correo : ', correo[i]
Escribir ' -----'
FinPara
Escribir ' Total de contactos: ', totalContactos
FinSi
3:
// --- BUSCAR CONTACTO ---
Definir busqueda Como Cadena
Definir j Como Entero
Definir encontrado Como Lógico
encontrado <- Falso
Si totalContactos=0 Entonces
Escribir ''
Escribir ' [INFO] No hay contactos registrados.'
SiNo
Escribir ''
Escribir ' -- Buscar Contacto --'
Escribir ' Ingrese nombre a buscar: 'Sin Saltar
Leer busqueda
Escribir ''
Escribir ' ===== RESULTADOS ====='
Para j<-1 Hasta totalContactos Con Paso 1 Hacer
// Simulación de coincidencia exacta de cadenas como pide el código C++
Si nombre[j]=busqueda Entonces
Escribir ' -----'
Escribir ' Nombre : ', nombre[j]
Escribir ' Telefono : ', telefono[j]
Escribir ' Correo : ', correo[j]
Escribir ' -----'
encontrado <- Verdadero
FinSi
FinPara
Si NO encontrado Entonces
Escribir ' [INFO] No se encontro ningun contacto con ese nombre.'
FinSi
FinSi
4:
Escribir ' Hasta luego.'
De Otro Modo:
Escribir ' Opción no válida.'
FinSegún
Hasta Que opcion=4
FinAlgoritmo

```

5. Diagrama de flujo



6. Codificación C++

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
using namespace std;
```

```
// Nombre del archivo donde se guardan los contactos
const string ARCHIVO = "agenda.txt";
```

```
// Clase Contacto
```

```
class Contacto {
private:
    string nombre;
    string telefono;
    string correo;
```

```
public:
```

```
    // Constructor por defecto
```

```
    Contacto() : nombre(""), telefono(""), correo("") {}
```

```
    // Constructor con parámetros
```

```
    Contacto(string _nombre, string _telefono, string _correo) {
        nombre = _nombre;
        telefono = _telefono;
        correo = _correo;
    }
```

```
    // Métodos GET
```

```
    string getNombre() const { return nombre; }
    string getTelefono() const { return telefono; }
    string getCorreo() const { return correo; }
```

```
    // Métodos SET
```

```
    void setNombre(string _nombre) { nombre = _nombre; }
    void setTelefono(string _telefono) { telefono = _telefono; }
    void setCorreo(string _correo) { correo = _correo; }
```

```

// Mostrar información del contacto
void mostrarContacto() const {
    cout << " -----" << endl;
    cout << " Nombre : " << nombre << endl;
    cout << " Telefono : " << telefono << endl;
    cout << " Correo : " << correo << endl;
    cout << " -----" << endl;
}

};

// Función: agregarContacto
// Guarda un contacto al final del archivo (modo append)
void agregarContacto() {
    string nombre, telefono, correo;

    cout << "\n -- Agregar Contacto --" << endl;
    cout << " Nombre : "; cin >> nombre;
    cout << " Telefono : "; cin >> telefono;
    cout << " Correo : "; cin >> correo;

    Contacto c(nombre, telefono, correo);

    // Abrir archivo en modo agregar (ios::app)
    ofstream archivo(ARCHIVO, ios::app);
    if (!archivo.is_open()) {
        cout << " [ERROR] No se pudo abrir el archivo." << endl;
        return;
    }

    // Guardar en formato: nombre|telefono|correo
    archivo << c.getNombre() << "|"
        << c.getTelefono() << "|"
        << c.getCorreo() << endl;

    archivo.close();
    cout << " [OK] Contacto guardado correctamente." << endl;
}

// Función: mostrarContactos
// Lee y muestra todos los contactos del archivo
void mostrarContactos() {
    ifstream archivo(ARCHIVO);
    if (!archivo.is_open()) {
        cout << " [INFO] No hay contactos registrados." << endl;
        return;
    }

    cout << "\n ===== LISTA DE CONTACTOS =====" << endl;

    string linea;
    int contador = 0;

    while (getline(archivo, linea)) {
        if (linea.empty()) continue;

        // Separar los campos por el delimitador '|'
        stringstream ss(linea);
        string nombre, telefono, correo;
        getline(ss, nombre, '|');
        getline(ss, telefono, '|');
        getline(ss, correo, '|');

        Contacto c(nombre, telefono, correo);

```

```

        c.mostrarContacto();
        contador++;
    }

    archivo.close();

    if (contador == 0) {
        cout << "    [INFO] No hay contactos registrados." << endl;
    } else {
        cout << "    Total de contactos: " << contador << endl;
    }
}

// Función: buscarContacto
// Busca un contacto por nombre (búsqueda parcial)
void buscarContacto() {
    string busqueda;
    cout << "\n  -- Buscar Contacto --" << endl;
    cout << "    Ingrese nombre a buscar: "; cin >> busqueda;

    ifstream archivo(ARCHIVO);
    if (!archivo.is_open()) {
        cout << "    [INFO] No hay contactos registrados." << endl;
        return;
    }

    string linea;
    bool encontrado = false;

    cout << "\n  ===== RESULTADOS =====" << endl;

    while (getline(archivo, linea)) {
        if (linea.empty()) continue;

        stringstream ss(linea);
        string nombre, telefono, correo;
        getline(ss, nombre, '|');
        getline(ss, telefono, '|');
        getline(ss, correo, '|');

        // Búsqueda insensible a mayúsculas no es requerida; búsqueda exacta
        if (nombre.find(busqueda) != string::npos) {
            Contacto c(nombre, telefono, correo);
            c.mostrarContacto();
            encontrado = true;
        }
    }

    archivo.close();

    if (!encontrado) {
        cout << "    [INFO] No se encontró ningún contacto con ese nombre." << endl;
    }
}

//Menú interactivo
int main() {
    int opcion;

    cout << "===== " << endl;
    cout << "    Agenda Telefónica (Ej.4)    " << endl;
    cout << "===== " << endl;

```

```

do {
    cout << "\n MENU PRINCIPAL" << endl;
    cout << " 1. Agregar contacto" << endl;
    cout << " 2. Mostrar contactos" << endl;
    cout << " 3. Buscar contacto" << endl;
    cout << " 4. Salir" << endl;
    cout << " Opcion: ";
    cin >> opcion;

    switch (opcion) {
        case 1: agregarContacto(); break;
        case 2: mostrarContactos(); break;
        case 3: buscarContacto(); break;
        case 4: cout << " Hasta luego." << endl; break;
        default: cout << " Opción no válida." << endl;
    }

    } while (opcion != 4);

    return 0;
}

```

7. Codificación en Java

```

import java.io.*;
import java.util.Scanner;

public class Ej4main {

    // Nombre del archivo donde se guardan los contactos
    private static final String ARCHIVO = "agenda.txt";
    private static Scanner sc = new Scanner(System.in);

    // Método: agregarContacto
    // Solicita datos y los guarda en el archivo (append)
    public static void agregarContacto() throws IOException {
        System.out.println("\n -- Agregar Contacto --");
        System.out.print(" Nombre : "); String nombre = sc.next();
        System.out.print(" Telefono : "); String telefono = sc.next();
        System.out.print(" Correo : "); String correo = sc.next();

        Ej4Contacto c = new Ej4Contacto(nombre, telefono, correo);

        // Abrir en modo append (true = agregar sin borrar contenido)
        FileWriter fw = new FileWriter(ARCHIVO, true);
        BufferedWriter bw = new BufferedWriter(fw);
        bw.write(c.toArchivo());
        bw.newLine();
        bw.close();
        fw.close();

        System.out.println(" [OK] Contacto guardado correctamente.");
    }

    // Método: mostrarContactos
    // Lee el archivo y muestra todos los contactos
    public static void mostrarContactos() throws IOException {
        File f = new File(ARCHIVO);
        if (!f.exists()) {
            System.out.println(" [INFO] No hay contactos registrados.");
            return;
        }

        System.out.println("\n ===== LISTA DE CONTACTOS =====");
    }
}

```

```

BufferedReader br = new BufferedReader(new FileReader(ARCHIVO));
String linea;
int contador = 0;

while ((linea = br.readLine()) != null) {
    if (linea.trim().isEmpty()) continue;
    Ej4Contacto c = Ej4Contacto.fromArchivo(linea);
    c.mostrarContacto();
    contador++;
}
br.close();

if (contador == 0) {
    System.out.println(" [INFO] No hay contactos registrados.");
} else {
    System.out.println(" Total de contactos: " + contador);
}
}

// Método: buscarContacto
// Busca contactos por nombre (búsqueda parcial)
public static void buscarContacto() throws IOException {
    System.out.println("\n -- Buscar Contacto --");
    System.out.print(" Ingrese nombre a buscar: ");
    String busqueda = sc.next();

    File f = new File(ARCHIVO);
    if (!f.exists()) {
        System.out.println(" [INFO] No hay contactos registrados.");
        return;
    }

    BufferedReader br = new BufferedReader(new FileReader(ARCHIVO));
    String linea;
    boolean encontrado = false;

    System.out.println("\n ===== RESULTADOS =====");

    while ((linea = br.readLine()) != null) {
        if (linea.trim().isEmpty()) continue;
        Ej4Contacto c = Ej4Contacto.fromArchivo(linea);
        if (c.getNombre().contains(busqueda)) {
            c.mostrarContacto();
            encontrado = true;
        }
    }
    br.close();

    if (!encontrado) {
        System.out.println(" [INFO] No se encontró ningún contacto con ese
nombre.");
    }
}

// Método principal - Menú interactivo
public static void main(String[] args) {
    System.out.println("=====");
    System.out.println(" Agenda Telefónica - Java (APE05 Ej.4) ");
    System.out.println("=====");

    int opcion;
    do {
        System.out.println("\n MENU PRINCIPAL");

```

```

        System.out.println(" 1. Agregar contacto");
        System.out.println(" 2. Mostrar contactos");
        System.out.println(" 3. Buscar contacto");
        System.out.println(" 4. Salir");
        System.out.print(" Opcion: ");
        opcion = sc.nextInt();

        try {
            switch (opcion) {
                case 1: agregarContacto(); break;
                case 2: mostrarContactos(); break;
                case 3: buscarContacto(); break;
                case 4: System.out.println(" Hasta luego."); break;
                default: System.out.println(" [ERROR] Opción no válida.");
            }
        } catch (IOException e) {
            System.out.println(" [ERROR] Problema con el archivo: " +
e.getMessage());
        }

        } while (opcion != 4);

        sc.close();
    }
}

```

8. Prueba Solución C++

```

PS C:\Users\Matias\Desktop\deberes uta\alg promg log\c++>
main }
=====
Agenda Telefonica C++ (Ej.4)
=====

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 1

-- Agregar Contacto --
Nombre : Giuly
Telefono : 0944218134
Correo : giuly@hotmail.com
Contacto guardado correctamente.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 1

-- Agregar Contacto --
Nombre : Elias
Telefono : 09898241563
Correo : elias@gmail.com
Contacto guardado correctamente.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 1

-- Agregar Contacto --
Nombre : Dominic
Telefono : 099482914521
Correo : dom@hotmail.com
Contacto guardado correctamente.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 2

```

```

===== LISTA DE CONTACTOS =====
-----
Nombre : Ricardo
Telefono : 0998184346
Correo : ricky@hotmail.com
-----
Nombre : Elias
Telefono : 0994284215
Correo : eliasolorzanco@gmail.com
-----
Nombre : Giuly
Telefono : 0994214521
Correo : giuly@gmail.com
-----
Nombre : Giuly
Telefono : 0944218134
Correo : giuly@hotmail.com
-----
Nombre : Elias
Telefono : 09898241563
Correo : elias@gmail.com
-----
Nombre : Dominic
Telefono : 099482914521
Correo : dom@hotmail.com
-----
Total de contactos: 6

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 3

-- Buscar Contacto --
Ingrese nombre a buscar: giuly

===== RESULTADOS =====
No se encontro ningun contacto con ese nombre.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 4
Hasta luego.
PS C:\Users\Matias\Desktop\deberes uta\alg promg log\c++>

```

9. Prueba de Solución en Java


```
PS C:\Users\Matias\Desktop\deberes_uta\alg_promg_log\Java>
n }
=====
Agenda Telefónica ? Java (APE05 Ej.4)
=====

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 1

-- Agregar Contacto --
Nombre : Matias
Telefono : 0998117410
Correo : mriv@gmail.com
[OK] Contacto guardado correctamente.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 1

-- Agregar Contacto --
Nombre : joseph
Telefono : 099482356
Correo : jlbunay@hotmail.com
[OK] Contacto guardado correctamente.

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 3

-- Buscar Contacto --
Ingrese nombre a buscar: joseph

===== RESULTADOS =====
-----
Nombre : joseph
Telefono : 099482356
Correo : jlbunay@hotmail.com
-----

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 2

===== LISTA DE CONTACTOS =====
-----
Nombre : Matias
Telefono : 0998117410
Correo : mriv@gmail.com
-----
Nombre : joseph
Telefono : 099482356
Correo : jlbunay@hotmail.com
-----
Total de contactos: 2

MENU PRINCIPAL
1. Agregar contacto
2. Mostrar contactos
3. Buscar contacto
4. Salir
Opcion: 4
Hasta luego.
PS C:\Users\Matias\Desktop\deberes_uta\alg_promg_log\Java>
```

Ejercicio 5 – Sistema de Notas (POO, Métodos y Persistencia en Archivos)

1. Análisis

El Ejercicio 5 constituye la implementación de un sistema de gestión y registro de rendimiento académico estudiantil con persistencia de datos en un archivo físico de texto plano. La solución de software aplica los principios de la Programación Orientada a Objetos (POO) y una arquitectura modularizada mediante métodos específicos para cubrir el flujo de procesamiento de calificaciones:

2. Declaración de Variables

Variable / Atributo	Tipo de Dato	Descripción/Funcionalidad
---------------------	--------------	---------------------------

ARCHIVO	String (Constante)	Identificador y ruta del archivo físico de persistencia ("notas.txt" o "notas_java.txt").
---------	--------------------	---

Variable / Atributo	Tipo de Dato	Descripción
nombre	String / Cadena	Atributo privado que guarda el nombre completo del estudiante evaluado.
nota1, nota2, nota3	double / Real	Variables numéricas de precisión para almacenar las tres calificaciones parciales.
promedio	double / Real	Atributo que almacena el valor numérico procesado del promedio aritmético de las notas.
condicion	String / Cadena	Estado cualitativo final determinado por el sistema ("Aprobado" o "Reprobado").
est	Objeto (Estudiante)	Instancia de la clase Estudiante creada dinámicamente en el método principal (main).
sc / teclado	Scanner / Stream	Objeto de flujo de entrada para la captura de datos desde la consola estándar de comandos.

3. Algoritmo-Explicacion

1. **Inicio del Programa:** Se reserva espacio en memoria para la instancia de procesamiento de la clase Estudiante.
2. **Entrada de Datos:** Solicitar y capturar secuencialmente por consola el nombre del estudiante, seguido de la Nota 1, Nota 2 y Nota 3 de sus parciales.
3. **Procesamiento de Calificaciones:** Invocar al método encargado de realizar la operación aritmética del promedio aritmético de las tres notas parciales.
4. **Evaluación Lógica de Condición:** Ejecutar el método de decisión que evalúa si el promedio es mayor o igual a 7.0 para asignar la cadena "Aprobado", o de lo contrario, fijar la condición de "Reprobado".
5. **Salida de Reporte (READ):** Desplegar en la interfaz de usuario de la consola el bloque

formalizado con el reporte del rendimiento académico del alumno.

6. **Persistencia de Datos:** Invocar al método de almacenamiento que abre el canal de escritura externa sobre el archivo físico de texto, escribe la cadena formateada con los delimitadores de registro y cierra la conexión del flujo de forma segura.
7. **Fin del Programa:** Liberación completa de los recursos del sistema de entrada/salida y cie



4. Pseudocódigo en PSeInt

Proceso SistemaNotas

Definir nombre, condicion Como Cadena
Definir notal, nota2, nota3, promedio Como Real

Escribir "=== REGISTRO DE NOTAS ==="

Escribir "Ingrese el nombre:"
Leer nombre

Escribir "Ingrese Nota 1:"
Leer notal

Escribir "Ingrese Nota 2:"
Leer nota2

Escribir "Ingrese Nota 3:"
Leer nota3

promedio <- (notal + nota2 + nota3) / 3

Si promedio >= 7 Entonces
 condicion <- "Aprobado"
SiNo
 condicion <- "Reprobado"
FinSi

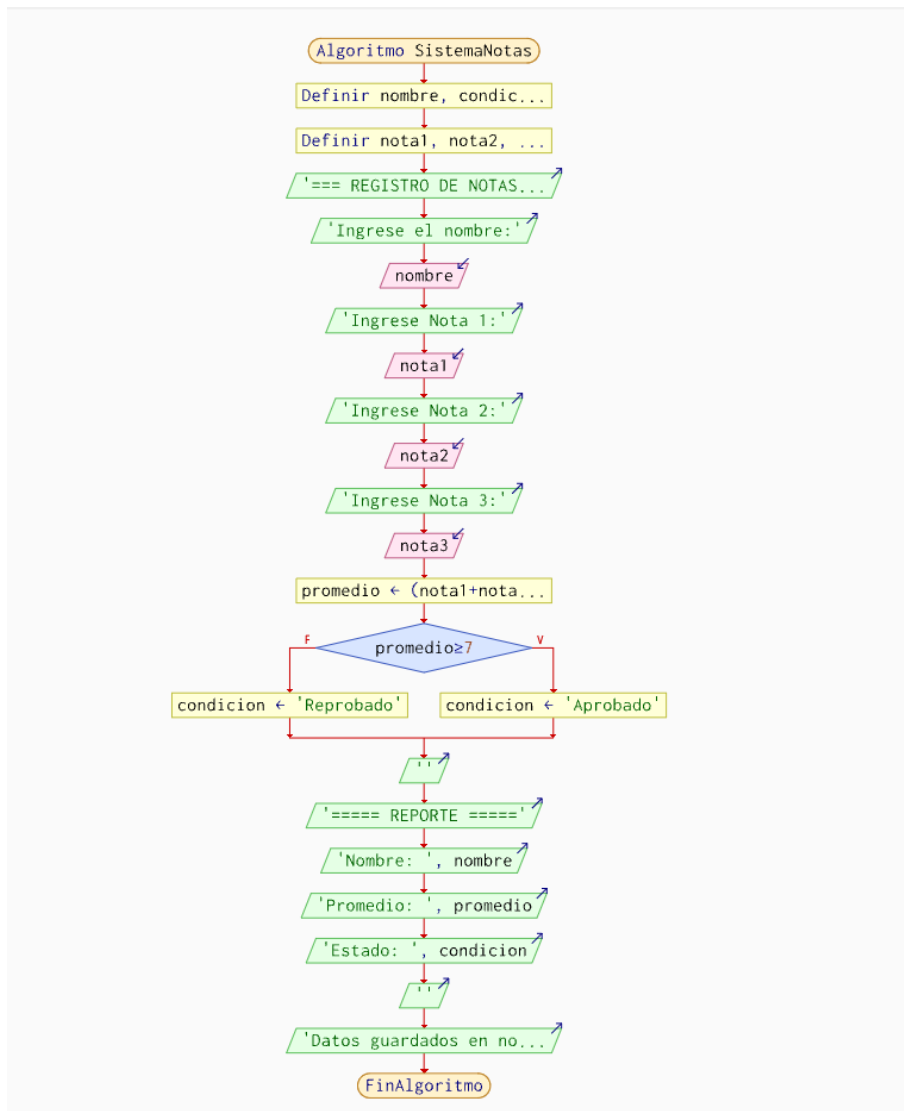
Escribir ""
Escribir "===== REPORTE ====="

Escribir "Nombre: ", nombre
Escribir "Promedio: ", promedio
Escribir "Estado: ", condicion

Escribir ""
Escribir "Datos guardados en notas.txt"



5. Diagrama de Flujo



6. Código Fuente en C++

```
// =====
// APE 05 - Ejercicio 5: Sistema de Calificaciones
// Sistema de Gestion de Notas (POO, Metodos y Archivos)
// Asignatura: Algoritmos y Logica de Programacion
// Docente: Jose Ruben Caiza
// =====

#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

class Estudiante {
public:
    string nombre;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
double nota1, nota2, nota3; // Atributos de las notas
double promedio;
string condicion;

// CONSTRUCTOR CORREGIDO: Ahora sí inicializa todas las variables
numéricas en 0.0
Estudiante() {
    nombre = "";
    nota1 = 0.0;
    nota2 = 0.0;
    nota3 = 0.0;
    promedio = 0.0;
    condicion = "";
}

// MÉTODO CORREGIDO: Se agregaron las líneas para que sí te pida las
notas por teclado
void registrarDatos() {
    cout << "=== Registrar Estudiante ===" << endl;
    cout << "Nombre del estudiante: ";
    getline(cin, nombre);
    cout << "Nota 1: ";
    cin >> nota1;
    cout << "Nota 2: ";
    cin >> nota2;
    cout << "Nota 3: ";
    cin >> nota3;
}

void calcularPromedio() {
    promedio = (nota1 + nota2 + nota3) / 3.0;
}

void evaluarCondicion() {
    if (promedio >= 7.0) {
        condicion = "Aprobado";
    } else {
        condicion = "Reprobado";
    }
}

void mostrarReporte() const {
    cout << "\n===== REPORTE =====" << endl;
    cout << " Estudiante: " << nombre << endl;
    cout << " Nota 1      : " << nota1 << endl;
    cout << " Nota 2      : " << nota2 << endl;
    cout << " Nota 3      : " << nota3 << endl;
    cout << " Promedio    : " << fixed << setprecision(2) << promedio <<
endl;
    cout << " Estado      : " << condicion << endl;
    cout << "-----" << endl;
}

void guardarEnArchivo(const string& nombreArchivo) {
    ofstream archivo;
    archivo.open(nombreArchivo, ios::app);

    if (archivo.is_open()) {
        archivo << nombre << "|" << nota1 << "|" << nota2 << "|" << nota3
<< "|"
        << fixed << setprecision(2) << promedio << "|" <<
condicion << "\n";
        archivo.close();
    }
}
```



```
        cout << "[!] Registro guardado exitosamente en '" <<
nombreArchivo << "'. " << endl;
    } else {
        cout << "Error: No se pudo acceder al archivo de persistencia."
<< endl;
    }
}
};

int main() {
    const string archivo = "notas.txt";
    Estudiante est;

    // Ejecución secuencial sin saltos
    est.registrarDatos();
    est.calcularPromedio();
    est.evaluarCondicion();
    est.mostrarReporte();
    est.guardarEnArchivo(archivo);

    return 0;
}
```

7. Prueba de Escritorio – C++

```
=== Registrar Estudiante ===
Nombre del estudiante: Hana
Nota 1: 7
Nota 2: 8
Nota 3: 9

===== REPORTE =====
Estudiante: Hana
Nota 1 : 7
Nota 2 : 8
Nota 3 : 9
Promedio : 8.00
Estado : Aprobado
-----
[!] Registro guardado exitosamente en 'notas.txt'.
Process returned 0 (0x0)   execution time : 5.553 s
Press any key to continue.
```

8. Código Fuente en Java

```
package com.mycompany.productos;

import java.util.Scanner;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.io.IOException;

class EstudianteDatos {

    private String nombre;

    private double notal, nota2, nota3;

    private double promedio;

    private String condicion;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public void registrarDatos() {

    Scanner sc = new Scanner(System.in);

    System.out.println("=== Registrar Estudiante ===");

    System.out.print("Nombre del estudiante: ");

    this.nombre = sc.nextLine();

    System.out.print("Nota 1: ");

    this.nota1 = sc.nextDouble();

    System.out.print("Nota 2: ");

    this.nota2 = sc.nextDouble();

    System.out.print("Nota 3: ");

    this.nota3 = sc.nextDouble();

}

public void calcularPromedio() {

    this.promedio = (this.nota1 + this.nota2 + this.nota3) / 3.0;

}

public void evaluarCondition() {

    if (this.promedio >= 7.0) {

        this.condicion = "Aprobado";

    } else {

        this.condicion = "Reprobado";

    }

}

public void mostrarReporte() {

    System.out.println("\n===== REPORTE =====");

    System.out.println("  Estudiante: " + this.nombre);

    System.out.println("  Nota 1      : " + this.nota1);
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        System.out.println("  Nota 2      : " + this.nota2);

        System.out.println("  Nota 3      : " + this.nota3);

        System.out.printf("  Promedio   : %.2f%n", this.promedio);

        System.out.println("  Estado      : " + this.condicion);

        System.out.println("-----");

    }

    public void guardarEnArchivo(String nombreArchivo) {

        try (FileWriter fw = new FileWriter(nombreArchivo, true);

            PrintWriter escritor = new PrintWriter(fw)) {

            escritor.println(this.nombre + "|" + this.nota1 + "|" +
this.nota2 + "|" +
                                this.nota3 + "|" + String.format("%.2f",
this.promedio) + "|" + this.condicion);

            System.out.println(" Registro guardado exitosamente en '" +
nombreArchivo + "'.");

        } catch (IOException e) {

            System.out.println("Error crítico en flujo de persistencia de
archivos: " + e.getMessage());

        }

    }

}

public class Estudiante {

    static final String ARCHIVO = "notas_java.txt";

    public static void main(String[] args) {

        EstudianteDatos est = new EstudianteDatos();

        // Ejecución de todo el flujo

        est.registrarDatos();

        est.calcularPromedio();

        est.evaluarCondition();

        est.mostrarReporte();

    }

}
```



```
est.guardarEnArchivo (ARCHIVO);
```

```
}
```

```
}
```

9. Prueba de Escritorio – Java

```
=== Registrar Estudiante ===
Nombre del estudiante: Gina
Nota 1: 10
Nota 2: 9
Nota 3: 9

===== REPORTE =====
Estudiante: Gina
Nota 1 : 10.0
Nota 2 : 9.0
Nota 3 : 9.0
Promedio : 9,33
Estado : Aprobado

-----
Registro guardado exitosamente en 'notas_java.txt'.

BUILD SUCCESS

-----
Total time: 10.717 s
Finished at: 2026-05-22T15:45:51-05:00
-----
```

Ejercicio 6 – Control de Asistencia

1. Análisis

El control de asistencia requiere la gestión de registros con nombre, fecha y estado, permitiendo tanto la escritura persistente en un archivo como la lectura para procesar el historial. La lógica se basa en el uso de ciclos para recorrer el archivo y contadores condicionales para detectar las faltas, aplicando validaciones para asegurar que los estados ingresados sean correctos (ej. "Presente" o "Ausente").

2. Declaración de Variables

Variable	Tipo de Dato	Propósito
nombre	string	Almacenar el nombre del estudiante
fecha	string	Almacenar la fecha de asistencia
estado	string	Almacenar "Presente" o "Ausente"
op	int	Almacenar la opción del menú
faltas	int	Acumular el total de ausencias

3. Algoritmo

Inicio

1. Definir vectores y contador en cero.
2. Mostrar menú: Registrar, Ver historial, Salir.



3. Si elige Registrar:
 - Pedir nombre, fecha y estado (validar presente/ausente).
 - Guardar datos en vectores y subir contador.
4. Si elige Ver historial:
 - Recorrer vectores con un ciclo.
 - Mostrar datos y sumar si hay "Ausente".
5. Repetir hasta Salir.

Fin

4. Seudocódigo

Algoritmo ControlAsistenciaSinArchivos

Definir nombres, fechas, estados Como Caracter

Definir op, contador, i, faltas Como Entero

Dimension nombres[100], fechas[100], estados[100]

contador <- 0

Repetir

 Escribir ""

 Escribir "1. Registrar | 2. Ver historial | 3. Salir"

 Leer op

Segun op Hacer

 1:

 Escribir "Nombre:"

 Leer nombres[contador]

 Escribir "Fecha (DD/MM/AAAA):"

 Leer fechas[contador]

 Repetir

 Escribir "Estado (Presente/Ausente):"

 Leer estados[contador]

 Hasta Que estados[contador] = "Presente" O estados[contador] =
"Ausente"

 contador <- contador + 1

 Escribir "Registrado correctamente."



2:

```
faltas <- 0

Escribir "--- Historial ---"

Para i <- 0 Hasta contador - 1 Hacer

    Escribir nombres[i], " | ", fechas[i], " | ", estados[i]

    Si estados[i] = "Ausente" Entonces

        faltas <- faltas + 1

    FinSi

FinPara

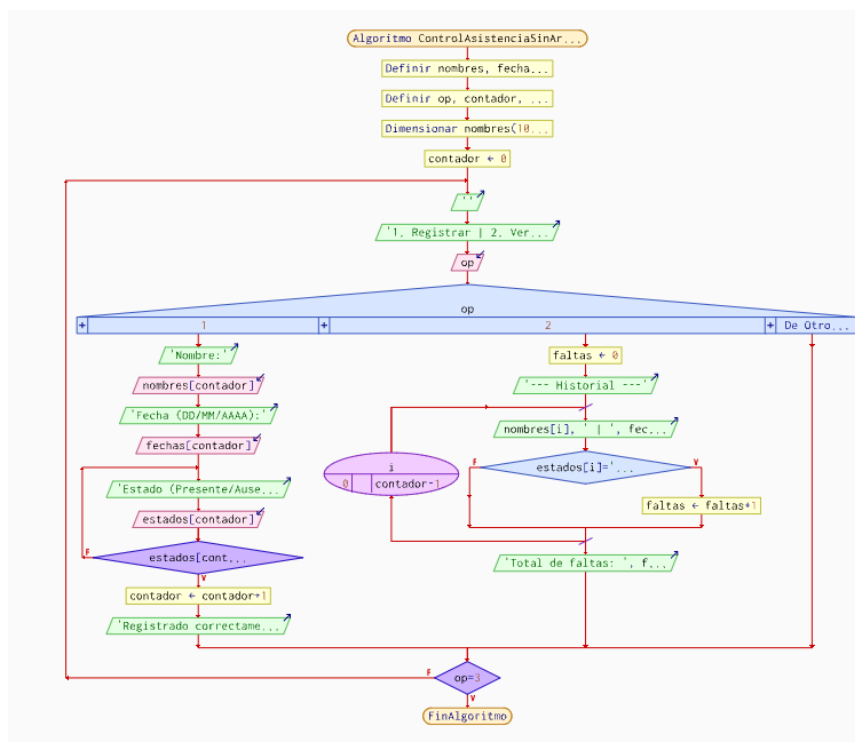
Escribir "Total de faltas: ", faltas

FinSegun

Hasta Que op = 3

FinAlgoritmo
```

5. Diagrama de Flujo



6. Código en C++

```
#include <iostream>

#include <fstream>

#include <string>

using namespace std;

void registrar() {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
string nombre, fecha, estado;

ofstream archivo("asistencia.txt", ios::app);

cout << "Nombre: "; cin >> nombre;

cout << "Fecha (DD/MM/AAAA): "; cin >> fecha;

do {

    cout << "Estado (Presente/Ausente): "; cin >> estado;

} while (estado != "Presente" && estado != "Ausente");

archivo << nombre << " " << fecha << " " << estado << endl;

archivo.close();

}

void historial() {

    string n, f, e;

    int faltas = 0;

    ifstream archivo("asistencia.txt");

    while (archivo >> n >> f >> e) {

        cout << n << " | " << f << " | " << e << endl;

        if (e == "Ausente") faltas++;

    }

    cout << "Total faltas: " << faltas << endl;

    archivo.close();

}

int main() {

    int op;

    do {

        cout << "\n1. Registrar | 2. Ver historial | 3. Salir: "; cin >> op;

        if (op == 1) registrar();

        else if (op == 2) historial();

    } while (op != 3);

    return 0;

}
```

7. Prueba de escritorio C++



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
1. Registrar | 2. Ver historial | 3. Salir: 1
Nombre: Andres
Fecha (DD/MM/AAAA): 18/05/2026
Estado (Presente/Ausente): Ausente
```

```
1. Registrar | 2. Ver historial | 3. Salir: 1
Nombre: Gina
Fecha (DD/MM/AAAA): 15/05/2026
Estado (Presente/Ausente): Presente
```

```
1. Registrar | 2. Ver historial | 3. Salir: 1
Nombre: Edison
Fecha (DD/MM/AAAA): 20/05/2026
Estado (Presente/Ausente): Ausente
```

```
1. Registrar | 2. Ver historial | 3. Salir: 2
Hanna | 18/05/2026 | Presente
Andres | 18/05/2026 | Ausente
Gina | 15/05/2026 | Presente
Edison | 20/05/2026 | Ausente
Total faltas: 2
```

```
1. Registrar | 2. Ver historial | 3. Salir: 3
PS C:\Users\hanna\Desktop\C++>
```

8. Código en Java

```
import java.io.*;

import java.util.Scanner;

public class ControlAsistencia {

    public static void main(String[] args) throws IOException {

        Scanner sc = new Scanner(System.in);

        int op;

        do {

            System.out.print("\n1. Registrar | 2. Historial | 3. Salir: ");

            op = sc.nextInt();

            if (op == 1) {

                try (PrintWriter w = new PrintWriter(new
                    FileWriter("asistencia.txt", true))) {

                    System.out.print("Nombre: "); String n = sc.next();

                    System.out.print("Fecha: "); String f = sc.next();

                    String e;

                    do { System.out.print("Estado (Presente/Ausente): "); e
                        = sc.next(); }

                    while (!e.equals("Presente") && !e.equals("Ausente"));

                    w.println(n + " " + f + " " + e);

                }

            } else if (op == 2) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
try (Scanner fs = new Scanner(new File("asistencia.txt"))) {  
  
    int faltas = 0;  
  
    while (fs.hasNext()) {  
  
        String n = fs.next(), f = fs.next(), e = fs.next();  
  
        System.out.println(n + " | " + f + " | " + e);  
  
        if (e.equals("Ausente")) faltas++;  
  
    }  
  
    System.out.println("Total faltas: " + faltas);  
  
    } catch (FileNotFoundException ex) { System.out.println("No  
hay registros."); }  
  
    }  
  
    } while (op != 3);  
  
}
```

9. Prueba de Escritorio Java

```
1. Registrar | 2. Historial | 3. Salir: 1  
Nombre: Andres  
Fecha: 22/06/2026  
Estado (Presente/Ausente): Presente  
  
1. Registrar | 2. Historial | 3. Salir: 1  
Nombre: Gina  
Fecha: 22/06/2026  
Estado (Presente/Ausente): Ausente  
  
1. Registrar | 2. Historial | 3. Salir: 1  
Nombre: Edison  
Fecha: 18/06/2026  
Estado (Presente/Ausente): Presente
```

```
1. Registrar | 2. Historial | 3. Salir: 2  
Andres | 17 | Presente  
Andres | 15 | Presente  
Andres | 22/05/2026 | Ausente  
Hanna | 22/06/2026 | Presente  
Andres | 22/06/2026 | Presente  
Gina | 22/06/2026 | Ausente  
Edison | 18/06/2026 | Presente  
Total faltas: 2
```

```
1. Registrar | 2. Historial | 3. Salir: 3  
PS C:\Users\hanna\Desktop\JAVA\EJERCICIO1 ESTUDIANTES>
```

Ejercicio 7 – Cajero Automático Básico



1. Análisis

El programa simula un cajero automático que permite a un usuario:

- Consultar saldo
- Depositar dinero
- Retirar dinero

2. Declaración de variables

Entrada

Variable	Tipo	Propósito	Ubicación
user	string	Nombre de usuario ingresado por teclado	main()
monto	double	Cantidad de dinero a depositar o retirar	main()
op	int	Opción del menú seleccionada	main()
linea	string	Línea leída desde el archivo	cargar()
u	string	Usuario temporal al leer archivo	cargar()
s	string	Saldo temporal como string	cargar()

Proceso

Variable	Tipo	Propósito	Ubicación
cuenta	CuentaBancaria	Objeto que representa la cuenta del usuario	main()
f	ifstream / ofstream	Flujo para leer/escribir archivo	cargar(), guardar()
ss	stringstream	Convierte línea en flujo para parsear	cargar()

Salida

Variable/Tipo	Propósito	Ubicación
saldo (double)	Saldo actual de la cuenta	consultarSaldo(), depositar(), retirar()
Mensajes en consola	Información de operaciones	Todas las funciones

3. Algoritmo

INICIO

1. Declarar variables:

- user: cadena
- op: entero
- monto: real

2. Mostrar "=== Cajero Automatico ==="

3. Solicitar al usuario: "Usuario: "

Leer y guardar en 'user'

4. Llamar a la función cargar(user) para obtener la cuenta:

4.1. Abrir archivo "cajero.txt" para lectura

4.2. Mientras haya líneas en el archivo:

4.2.1. Leer una línea

4.2.2. Separar la línea por '|' (usuario y saldo)



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



- 4.2.3. Si el usuario leído coincide con 'user':
- Crear y devolver cuenta con ese saldo
- 4.3. Si no se encontró, devolver cuenta nueva con saldo = 0
5. Repetir (hasta que op = 0):
- 5.1. Mostrar menú:
- "1. Consultar saldo"
 - "2. Depositar"
 - "3. Retirar"
 - "0. Salir"
 - "Opcion: "
- 5.2. Leer 'op'
- 5.3. Si op == 1:
- llamar a consultarSaldo()
- 5.4. Si op == 2:
- solicitar "Monto: "
 - leer 'monto'
 - llamar a depositar(monto)
- 5.5. Si op == 3:
- solicitar "Monto: "
 - leer 'monto'
 - llamar a retirar(monto)
6. FIN (cuando op == 0)
- FUNCIONES SECUNDARIAS:**

FUNCION cargar(user):

- Abrir archivo "cajero.txt"
- Leer línea por línea
- Extraer usuario y saldo (formato: "usuario|saldo")
- Si coincide con user, devolver CuentaBancaria(usuario, saldo)
- Si no, devolver CuentaBancaria(user, 0.0)

FUNCION guardar():

- Abrir archivo "cajero.txt" (soescribe todo)
- Escribir "usuario|saldo"

FUNCION consultarSaldo():

- Mostrar "Saldo actual: \$saldo"

FUNCION depositar(monto):

- Si monto <= 0: Mostrar "Monto invalido." y retornar
- saldo = saldo + monto
- Llamar a guardar()
- Mostrar "Deposito exitoso. Saldo: \$saldo"



FUNCION retirar(monto):

- Si monto ≤ 0 o monto $>$ saldo:
Mostrar "Fondos insuficientes o monto invalido." y retornar
- saldo = saldo - monto
- Llamar a guardar()
- Mostrar "Retiro exitoso. Saldo: \$saldo"

FIN

4. Pseudo código en pseint

Algoritmo CajeroAutomatico

```
// Declaración de variables
Definir usuario, linea, u, s Como Cadena
Definir saldo, monto Como Real
Definir opcion, encontrado, i Como Entero
Definir archivo Como Cadena

// Entrada
Escribir "=== Cajero Automatico ==="
Escribir "Usuario: "
Leer usuario

// Proceso: cargar saldo del archivo
archivo <- "cajero.txt"
saldo <- 0
encontrado <- 0

// Simulación de lectura de archivo
Escribir "Leyendo archivo ", archivo
// En PSeInt real, se usaría la función LeerArchivo
// Para este ejercicio, simulamos con datos de ejemplo

// Simulación: preguntar si es nuevo o existente
Escribir "¿Es un usuario nuevo? (1=Si, 0=No): "
Leer encontrado

Si encontrado = 0 Entonces
    Escribir "Ingrese saldo inicial: "
    Leer saldo
Sino
    saldo <- 0
FinSi

// Menú principal
Repetir
    Escribir ""
    Escribir "1. Consultar saldo"
    Escribir "2. Depositar"
    Escribir "3. Retirar"
    Escribir "0. Salir"
    Escribir "Opcion: "
    Leer opcion

    Segun opcion Hacer
        Caso 1: // Consultar saldo
            Escribir "Saldo actual: $", saldo

        Caso 2: // Depositar
            Escribir "Monto: "
            Leer monto
            Si monto  $\leq 0$  Entonces
                Escribir "Monto invalido."
            Sino
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        saldo <- saldo + monto
        Escribir "Deposito exitoso. Saldo: $", saldo
    FinSi

    Caso 3: // Retirar
        Escribir "Monto: "
        Leer monto
        Si monto <= 0 O monto > saldo Entonces
            Escribir "Fondos insuficientes o monto invalido."
        Sino
            saldo <- saldo - monto
            Escribir "Retiro exitoso. Saldo: $", saldo
        FinSi

    Caso 0: // Salir
        Escribir "Gracias por usar el cajero"

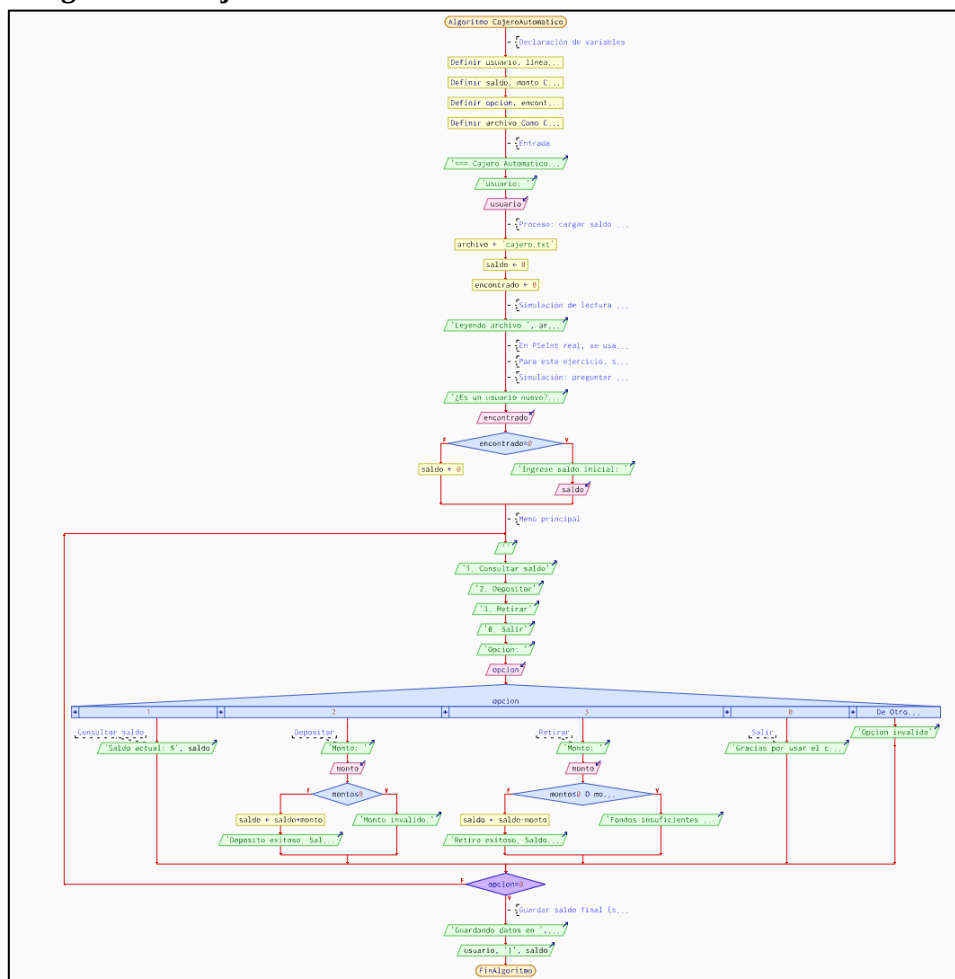
    De Otro Modo:
        Escribir "Opcion invalida"
    FinSegun

Hasta Que opcion = 0

// Guardar saldo final (simulado)
Escribir "Guardando datos en ", archivo
Escribir usuario, "|", saldo

FinAlgoritmo
```

5. Diagrama de flujo





6. Código C++

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
using namespace std;

class CuentaBancaria {
private:
    string usuario; double saldo;
public:
    CuentaBancaria(string u, double s) : usuario(u), saldo(s) {}

    // Carga saldo desde el archivo
    static CuentaBancaria cargar(const string& user) {
        ifstream f("cajero.txt"); string linea;
        while (getline(f,linea)) {
            stringstream ss(linea); string u,s;
            getline(ss,u,'|'); getline(ss,s,'|');
            if (u == user) return CuentaBancaria(u, stod(s));
        }
        return CuentaBancaria(user, 0.0); // cuenta nueva
    }
    void guardar() {
        ofstream f("cajero.txt"); // sobreescribe
        f << usuario << "|" << saldo << "\n";
    }
    void consultarSaldo() {
        cout << "Saldo actual: $" << saldo << endl;
    }
    void depositar(double monto) {
        if (monto <= 0) { cout << "Monto invalido." << endl; return; }
        saldo += monto;
        guardar();
        cout << "Deposito exitoso. Saldo: $" << saldo << endl;
    }
    void retirar(double monto) {
        if (monto <= 0 || monto > saldo) {
            cout << "Fondos insuficientes o monto invalido." << endl;
            return;
        }
        saldo -= monto;
        guardar();
        cout << "Retiro exitoso. Saldo: $" << saldo << endl;
    }
};

int main() {
    string user;
    cout << "=== Cajero Automatico ===" << endl;
    cout << "Usuario: "; cin >> user;
    CuentaBancaria cuenta = CuentaBancaria::cargar(user);
    int op; double monto;
    do {
        cout << "\n1. Consultar saldo\n2. Depositar"
              << "\n3. Retirar\n0. Salir\nOpcion: ";
        cin >> op;
        if (op == 1) cuenta.consultarSaldo();
        else if (op == 2) { cout << "Monto: "; cin >> monto;
            cuenta.depositar(monto); }
        else if (op == 3) { cout << "Monto: "; cin >> monto;
            cuenta.retirar(monto); }
        } while (op != 0);
    return 0;
}
```



7. Prueba de escritorios C++

```
=== Cajero Automatico ===
Usuario: 12345

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir
Opcion: 1
Saldo actual: $0

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir
Opcion: 2
Monto: 100000
Deposito exitoso. Saldo: $100000

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir
Opcion: 3
Monto: 5000
Retiro exitoso. Saldo: $95000

1. Consultar saldo
2. Depositar
3. Retirar
0. Salir
Opcion: 0
```

8. Código En Java

```
import java.io.*;
import java.util.Scanner;

class CuentaBancaria {
    private String usuario;
    private double saldo;

    // Constructor
    public CuentaBancaria(String usuario, double saldo) {
        this.usuario = usuario;
        this.saldo = saldo;
    }

    // Carga saldo desde el archivo
    public static CuentaBancaria cargar(String user) {
        String archivo = "cajero.txt";

        try (BufferedReader br = new BufferedReader(new
        FileReader(archivo))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] partes = linea.split("\\|");
                if (partes.length == 2 && partes[0].equals(user)) {
                    double saldo = Double.parseDouble(partes[1]);
                    return new CuentaBancaria(user, saldo);
                }
            }
        } catch (FileNotFoundException e) {
            // Archivo no existe, se creará nuevo
        } catch (IOException e) {
            System.err.println("Error al leer archivo: " + e.getMessage());
        }

        return new CuentaBancaria(user, 0.0); // Cuenta nueva
    }

    // Guarda el saldo actual en el archivo
    public void guardar() {
        try (PrintWriter pw = new PrintWriter(new
        FileWriter("cajero.txt"))) {

```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        pw.println(usuario + "|" + saldo);
    } catch (IOException e) {
        System.err.println("Error al guardar: " + e.getMessage());
    }
}

// Consulta saldo
public void consultarSaldo() {
    System.out.printf("Saldo actual: $%.2f%n", saldo);
}

// Depositar
public void depositar(double monto) {
    if (monto <= 0) {
        System.out.println("Monto invalido.");
        return;
    }
    saldo += monto;
    guardar();
    System.out.printf("Deposito exitoso. Saldo: $%.2f%n", saldo);
}

// Retirar
public void retirar(double monto) {
    if (monto <= 0 || monto > saldo) {
        System.out.println("Fondos insuficientes o monto invalido.");
        return;
    }
    saldo -= monto;
    guardar();
    System.out.printf("Retiro exitoso. Saldo: $%.2f%n", saldo);
}
}

public class CajeroAutomatico {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Cajero Automatico ===");
        System.out.print("Usuario: ");
        String user = sc.nextLine();

        CuentaBancaria cuenta = CuentaBancaria.cargar(user);

        int opcion;
        double monto;

        do {
            System.out.println("\n1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("0. Salir");
            System.out.print("Opcion: ");
            opcion = sc.nextInt();

            switch (opcion) {
                case 1:
                    cuenta.consultarSaldo();
                    break;
                case 2:
                    System.out.print("Monto: ");
                    monto = sc.nextDouble();
                    cuenta.depositar(monto);
                    break;
                case 3:
                    System.out.print("Monto: ");
                    monto = sc.nextDouble();
            }
        }
    }
}
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Al iniciar, carga los libros existentes; al registrar, guarda de inmediato para garantizar persistencia. Aplica POO con la clase Libro y métodos estáticos/funciones para cada operación.

2. Declaración de Variables

Variable	Tipo	Descripción
NOMBRE_ARCHIVO	String (const)	Nombre del archivo de almacenamiento de libros
codigo	String	Código único identificador del libro
titulo	String	Título del libro
autor	String	Nombre del autor del libro
estado	String	Estado del libro: "disponible" o "prestado"
busqueda	String	Término ingresado por el usuario para buscar
opcion	int / Entero	Opción seleccionada en el menú
contDisponibles	int / Entero	Contador de libros con estado disponible
contPrestados	int / Entero	Contador de libros con estado prestado
libros	vector / List	Colección principal de todos los objetos Libro
linea	String	Variable temporal para lectura de líneas del archivo

3. Algoritmo

1. Al iniciar, abrir "libros.txt" y cargar todos los libros existentes en la colección.
2. Mostrar el menú de opciones.
3. Leer la opción ingresada por el usuario.
4. Si opción = 1: solicitar código, título, autor y estado; validar estado en bucle; agregar libro a la colección; guardar toda la colección en el archivo.
5. Si opción = 2: leer criterio de búsqueda; recorrer la colección comparando (sin distinción de mayúsculas) con código y título; mostrar los coincidentes.



6. Si opción = 3: recorrer la colección dos veces, mostrando primero los disponibles y luego los prestados, con contadores al final.
7. Si opción = 4: recorrer la colección completa y mostrar cada libro numerado.
8. Si opción = 0: terminar el programa.
9. Para cualquier otra opción: mostrar mensaje de opción inválida.
10. Repetir desde el paso 2 hasta que la opción sea 0.

4. Pseudocódigo en PSeInt

Proceso BibliotecaVirtual

Definir codigo, titulo, autor, estado, busqueda Como Cadena

Definir opcion, contDisp, contPrest Como Entero

LeerLibros("libros.txt")

Repetir

 Escribir "===== BIBLIOTECA VIRTUAL ====="

 Escribir "1. Registrar libro"

 Escribir "2. Buscar libro"

 Escribir "3. Ver disponibles y prestados"

 Escribir "4. Ver catalogo completo"

 Escribir "0. Salir"

 Leer opcion

Segun opcion Hacer

 1:

 Leer codigo, titulo, autor

 Repetir

 Leer estado

 Si estado <> "disponible" Y estado <> "prestado" Entonces

 Escribir "Estado invalido"

 FinSi

 Hasta Que estado = "disponible" O estado = "prestado"

 AgregarLibro(codigo, titulo, autor, estado)

 GuardarLibros("libros.txt")

 Escribir "Libro registrado correctamente"



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



2:

```
Leer busqueda

Para i <- 0 Hasta largo(libros)-1 Hacer

    Si libros[i].codigo Contiene busqueda O

        libros[i].titulo Contiene busqueda Entonces

            MostrarLibro(libros[i])

        FinSi

    FinPara
```

3:

```
contDisp <- 0 ; contPrest <- 0

Para i <- 0 Hasta largo(libros)-1 Hacer

    Si libros[i].estado = "disponible" Entonces

        MostrarLibro(libros[i])

        contDisp <- contDisp + 1

    FinSi

FinPara

Para i <- 0 Hasta largo(libros)-1 Hacer

    Si libros[i].estado = "prestado" Entonces

        MostrarLibro(libros[i])

        contPrest <- contPrest + 1

    FinSi

FinPara

Escribir "Disponibles: ", contDisp, " | Prestados: ",
contPrest
```

4:

```
Para i <- 0 Hasta largo(libros)-1 Hacer

    MostrarLibro(libros[i])

FinPara

De Otro Modo:

    Escribir "Opcion invalida"

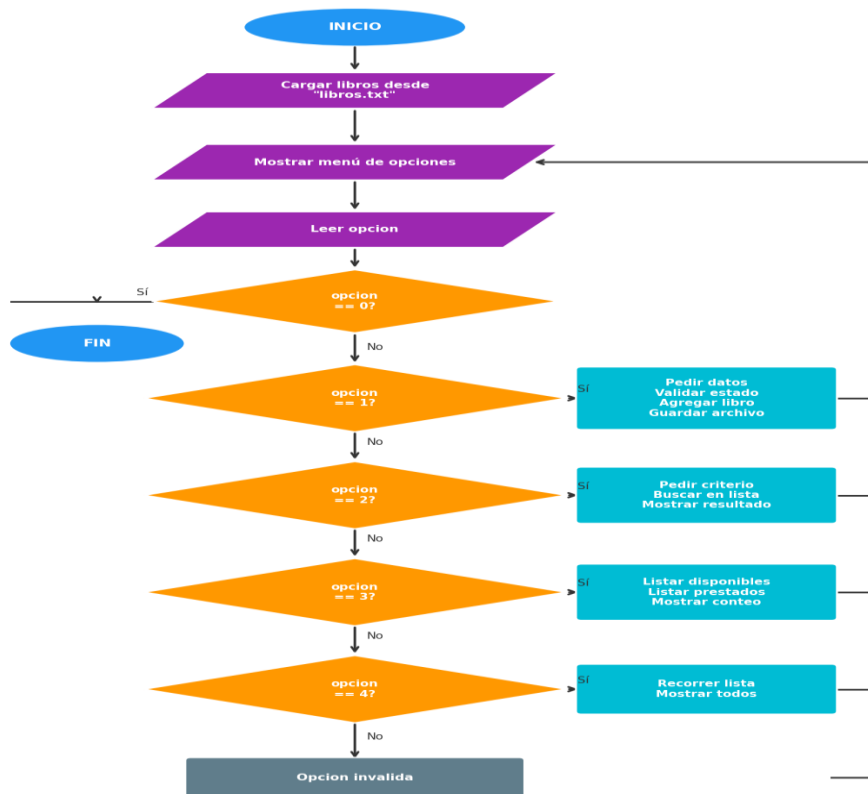
FinSegun

Hasta Que opcion = 0
```



FinProceso

5. Diagrama de Flujo



6. Código C++

```
// =====
// APE 05 - Ejercicio 8: Biblioteca Virtual
// Asignatura: Algoritmos y Logica de Programacion
// Docente: Jose Ruben Caiza
// =====
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
#include <iostream>

#include <fstream>

#include <string>

#include <vector>

#include <sstream>

#include <algorithm>

using namespace std;

class Libro {

public:

    string codigo, titulo, autor, estado;

    Libro(string c,string t,string a,string e):
    codigo(c),titulo(t),autor(a),estado(e){}

    void mostrar() {

        cout<<"  Codigo : "<<codigo<<endl;

        cout<<"  Titulo : "<<titulo<<endl;

        cout<<"  Autor   : "<<autor <<endl;

        cout<<"  Estado : "<<estado<<endl;

    }

    string toLinea(){ return codigo+"|"+titulo+"|"+autor+"|"+estado; }

};

vector<string> dividir(string l,char d){ vector<string> p; stringstream ss(l);
string t;

    while(getline(ss,t,d)) p.push_back(t); return p; }

vector<Libro> leerLibros(string f){ vector<Libro> lb; ifstream a(f);

    if(!a.is_open()) return lb;

    string ln; while(getline(a,ln)) if(!ln.empty()){ auto d=dividir(ln,'|');
if(d.size()==4) lb.push_back(Libro(d[0],d[1],d[2],d[3])); }

    a.close(); return lb; }

void guardarLibros(string f,vector<Libro>& lb){ ofstream a(f); for(auto& l:lb)
a<<l.toLinea()<<"\n"; }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
void registrarLibro(string f,vector<Libro>& lb){

    string c,t,a,e;

    cout<<"\n--- Registrar Nuevo Libro ---\nCodigo : "; cin>>c; cin.ignore();

    cout<<"Titulo : "; getline(cin,t);

    cout<<"Autor : "; getline(cin,a);

    do{ cout<<"Estado (disponible/prestado): "; getline(cin,e);

        if(e!="disponible"&&e!="prestado") cout<<"Estado invalido.\n";

    }while(e!="disponible"&&e!="prestado");

    lb.push_back(Libro(c,t,a,e)); guardarLibros(f,lb);

    cout<<"Libro registrado correctamente.\n";

}

void buscarLibro(vector<Libro>& lb){

    string b; cin.ignore(); cout<<"\nCriterio de busqueda: "; getline(cin,b);

    string bl=b; transform(bl.begin(),bl.end(),bl.begin(),::tolower);

    bool ok=false; cout<<"\n--- Resultados ---\n";

    for(auto& l:lb){

        string cl=l.codigo, tl=l.titulo;

        transform(cl.begin(),cl.end(),cl.begin(),::tolower);

        transform(tl.begin(),tl.end(),tl.begin(),::tolower);

        if(cl.find(bl)!=string::npos||tl.find(bl)!=string::npos){ l.mostrar();

        cout<<"-----\n"; ok=true; }

    }

    if(!ok) cout<<"Sin resultados.\n";

}

void mostrarPorEstado(vector<Libro>& lb){

    cout<<"\n===== DISPONIBLES =====\n"; int d=0;

    for(auto& l:lb) if(l.estado=="disponible"){ l.mostrar(); cout<<"---\n";

    d++; }

    if(!d) cout<<"Ninguno.\n";

    cout<<"\n===== PRESTADOS =====\n"; int p=0;

    for(auto& l:lb) if(l.estado=="prestado"){ l.mostrar(); cout<<"---\n"; p++;

    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
if(!p) cout<<"Ninguno.\n";

cout<<"Resumen: "<<d<<" disponible(s), "<<p<<" prestado(s).\n";

}

void mostrarTodos(vector<Libro>& lb){

    if(lb.empty()){ cout<<"Sin libros.\n"; return; }

    cout<<"\n===== CATALOGO =====\n";

    for(int i=0;i<(int)lb.size();i++){ cout<<"Libro #"<<i+1<<endl;
lb[i].mostrar(); cout<<"--\n"; }

    cout<<"Total: "<<lb.size()<<" libro(s).\n";

}

int main(){

    string f="libros.txt"; auto lb=leerLibros(f); int op;

    do{

        cout<<"\n===== BIBLIOTECA VIRTUAL =====\n1. Registrar libro\n2. Buscar
libro\n";

        cout<<"3. Ver disponibles y prestados\n4. Ver catalogo\n0.
Salir\nOpcion: "; cin>>op;

        switch(op){

            case 1: registrarLibro(f,lb); break;

            case 2: buscarLibro(lb); break;

            case 3: mostrarPorEstado(lb); break;

            case 4: mostrarTodos(lb); break;

            case 0: cout<<"Saliendo...\n"; break;

            default: cout<<"Invalido.\n";

        }

    }while(op!=0);

    return 0;

}
```

7. Prueba de Escritorio – C++

Ejecución del sistema registrando libros, buscando y mostrando por estado:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
==== BIBLIOTECA VIRTUAL ====
1. Registrar libro
2. Buscar libro
3. Ver disponibles y prestados
4. Ver catalogo completo
0. Salir
Opcion: 1

--- Registrar Nuevo Libro ---
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor  : J.R.R. Tolkien
Estado (disponible / prestado): disponible
Libro registrado y guardado correctamente.

Opcion: 2

Ingrese codigo o titulo a buscar: Tolkien

--- Resultados de Búsqueda ---
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor  : J.R.R. Tolkien
Estado : disponible

Opcion: 3

==== LIBROS DISPONIBLES ====
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor  : J.R.R. Tolkien
Estado : disponible

==== LIBROS PRESTADOS =====
Codigo : LIB002
Titulo : Cien Anios de Soledad
Autor  : Gabriel Garcia Marquez
Estado : prestado

Resumen: 1 disponible(s), 1 prestado(s).

Opcion: 0
Saliendo del sistema...
```

8. Código Java

```
// =====

// APE 05 - Ejercicio 8: Biblioteca Virtual

// Asignatura: Algoritmos y Logica de Programacion

// Docente: Jose Ruben Caiza

// =====

import java.io.*;

import java.util.*;

class Libro {

    private String codigo, titulo, autor, estado;

    public Libro(String c,String t,String a,String e){
        codigo=c;titulo=t;autor=a;estado=e; }

    public String getCodigo(){ return codigo; }

    public String getTitulo(){ return titulo; }

    public String getAutor() { return autor; }

    public String getEstado(){ return estado; }

    public void setEstado(String e){ estado=e; }

    public void mostrar(){

        System.out.println("  Codigo : "+codigo);

        System.out.println("  Titulo : "+titulo);

        System.out.println("  Autor  : "+autor);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println(" Estado : "+estado);

}

public String toLinea(){ return codigo+"|"+titulo+"|"+autor+"|"+estado; }

}

public class Ejercicio8 {

    static final String ARCHIVO = "libros.txt";

    static List<Libro> libros = new ArrayList<>();

    static Scanner sc = new Scanner(System.in);

    public static void leerLibros(){

        libros.clear();

        try(BufferedReader br=new BufferedReader(new FileReader(ARCHIVO))){

            String l; while((l=br.readLine())!=null) if(!l.isEmpty()){

                String[] d=l.split("\\|"); if(d.length==4) libros.add(new

Libro(d[0],d[1],d[2],d[3]));

            }

        }catch(IOException e){ /* primer uso */ }

    }

    public static void guardarLibros(){

        try(BufferedWriter bw=new BufferedWriter(new FileWriter(ARCHIVO))){

            for(Libro l:libros){ bw.write(l.toLinea()); bw.newLine(); }

        }catch(IOException e){ System.out.println("Error: "+e.getMessage()); }

    }

    public static void registrarLibro(){

        System.out.println("\n--- Registrar Nuevo Libro ---");

        System.out.print("Codigo : "); String c=sc.nextLine();

        System.out.print("Titulo : "); String t=sc.nextLine();

        System.out.print("Autor : "); String a=sc.nextLine();

        String e;
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
do{ System.out.print("Estado (disponible/prestado): ");
e=sc.nextLine();

        if(!e.equals("disponible")&&!e.equals("prestado"))
System.out.println("Estado invalido.");

        }while(!e.equals("disponible")&&!e.equals("prestado"));

        libros.add(new Libro(c,t,a,e)); guardarLibros();

        System.out.println("Libro registrado correctamente.");

    }

    public static void buscarLibro(){

        System.out.print("\nCriterio de busqueda: "); String
b=sc.nextLine().toLowerCase();

        boolean ok=false; System.out.println("\n--- Resultados ---");

        for(Libro l:libros)
if(l.getCodigo().toLowerCase().contains(b)||l.getTitulo().toLowerCase().contai
ns(b)){

            l.mostrar(); System.out.println("-----"); ok=true;

        }

        if(!ok) System.out.println("Sin resultados.");

    }

    public static void mostrarPorEstado(){

        System.out.println("\n===== DISPONIBLES ====="); int d=0;

        for(Libro l:libros) if(l.getEstado().equals("disponible")){
l.mostrar(); System.out.println("---"); d++; }

        if(d==0) System.out.println("Ninguno.");

        System.out.println("\n===== PRESTADOS ====="); int p=0;

        for(Libro l:libros) if(l.getEstado().equals("prestado")){ l.mostrar();
System.out.println("---"); p++; }

        if(p==0) System.out.println("Ninguno.");

        System.out.println("Resumen: "+d+" disponible(s), "+p+"
prestado(s).");

    }

    public static void mostrarTodos(){

        if(libros.isEmpty()){ System.out.println("Sin libros."); return; }

        System.out.println("\n===== CATALOGO =====");

        for(int i=0;i<libros.size();i++){ System.out.println("Libro #"+(i+1));
libros.get(i).mostrar(); System.out.println("---"); }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("Total: "+libros.size()+" libro(s).");
}

public static void main(String[] args){

    leerLibros(); int op;

    do{

        System.out.println("\n===== BIBLIOTECA VIRTUAL =====");

        System.out.println("1. Registrar libro\n2. Buscar libro\n3. Ver disponibles y prestados\n4. Ver catalogo\n0. Salir");

        System.out.print("Opcion: ");

        try{ op=Integer.parseInt(sc.nextLine());
        }catch(NumberFormatException e){ op=-1; }

        switch(op){

            case 1: registrarLibro(); break;

            case 2: buscarLibro(); break;

            case 3: mostrarPorEstado(); break;

            case 4: mostrarTodos(); break;

            case 0: System.out.println("Saliendo..."); break;

            default: System.out.println("Invalido.");

        }

    }while(op!=0);

    sc.close();

}
```

9. Prueba de Escritorio – Java

Ejecución con javac / java produciendo la misma salida que la versión C++:

```
===== BIBLIOTECA VIRTUAL =====
1. Registrar libro
2. Buscar libro
3. Ver disponibles y prestados
4. Ver catalogo completo
0. Salir
Opcion: 1

... Registrar Nuevo Libro ...
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor : J.R.R. Tolkien
Estado (disponible / prestado): disponible
Libro registrado y guardado correctamente.

Opcion: 2

Ingrese codigo o titulo a buscar: Tolkien

... Resultados de Búsqueda ...
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor : J.R.R. Tolkien
Estado : disponible

Opcion: 3

===== LIBROS DISPONIBLES =====
Codigo : LIB001
Titulo : El Senor de los Anillos
Autor : J.R.R. Tolkien
Estado : disponible

===== LIBROS PRESTADOS =====
Codigo : LIB002
Titulo : Cien Anios de Soledad
Autor : Gabriel Garcia Marquez
Estado : prestado

Resumen: 1 disponible(s), 1 prestado(s).

Opcion: 0
Saliendo del sistema...
```



Ejercicio 9 – Sistema de Ventas

1. Analisis

Desarrollar un programa en C++ y Java que permita registrar ventas de productos aplicando POO y manejo de archivos. El sistema debe solicitar el producto, cantidad y precio unitario, calcular el total automáticamente, generar una factura y guardar el registro en ventas.txt. Además debe mostrar el historial completo de ventas con el total de ingresos acumulados.

2. Declarar Variables

Entrada: nombre del producto, cantidad (entero > 0), precio unitario (double > 0). Opción del menú.

Proceso:

Clase Venta con atributos privados: producto, cantidad, precio, total

Método calcularTotal() que ejecuta total = cantidad * precio

Método mostrarFactura() para imprimir los datos de la venta

Método toArchivo() para serializar: producto|cantidad|precio|total

Método fromArchivo() para reconstruir el objeto desde el archivo

Variable totalGeneral (double) para acumular ingresos en el historial

Variable contador (int) para contar ventas registradas

Archivo ventas.txt en modo append para guardar cada venta

Salida: Factura generada al registrar. Historial de ventas con total de ingresos. Mensajes de error si los datos son inválidos.

3. Algoritmo

Inicio

Mostrar menú con opciones: Registrar venta / Historial / Salir.

Leer opción del usuario.

Si la opción es Registrar venta:

Pedir nombre del producto.

Pedir cantidad y validar que sea mayor a 0; repetir si no es válida.

Pedir precio unitario y validar que sea mayor a 0; repetir si no es válido.

Crear objeto Venta y llamar a calcularTotal().

Mostrar la factura generada con mostrarFactura().

Abrir ventas.txt en modo append.

Guardar: producto|cantidad|precio|total y salto de línea.

Cerrar el archivo y mostrar confirmación.

Si la opción es Historial de ventas:

Verificar si ventas.txt existe.

Leer línea por línea y reconstruir cada Venta con fromArchivo().

Llamar a mostrarFactura() por cada registro.

Acumular el total en totalGeneral.

Mostrar al final: número de ventas e ingresos totales.

Repetir hasta que el usuario elija Salir.

Fin

4. Pseudocodigo

```
Algoritmo SistemaVentas
// Límite máximo de almacenamiento en memoria para emular el archivo
Definir MAX_VENTAS Como Entero
MAX_VENTAS <- 100
// Arreglos paralelos que reemplazan los atributos de la clase Venta
Definir producto Como Cadena
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Definir cantidad Como Entero
Definir precio, total Como Real
Dimensionar producto(MAX_VENTAS)
Dimensionar cantidad(MAX_VENTAS)
Dimensionar precio(MAX_VENTAS)
Dimensionar total(MAX_VENTAS)
// Contador real de ventas registradas en el "archivo"
Definir totalVentas Como Entero
totalVentas <- 0
Definir opcion Como Entero
Escribir '===== '
Escribir ' Sistema de Ventas PSeInt (APE05 Ej.9) '
Escribir '===== '
Repetir
    Escribir ''
    Escribir ' MENU PRINCIPAL'
    Escribir ' 1. Registrar venta'
    Escribir ' 2. Historial de ventas'
    Escribir ' 3. Salir'
    Escribir ' Opcion: 'Sin Saltar
    Leer opcion
    Según opcion Hacer
        1:
            // --- REGISTRAR VENTA ---
            Si totalVentas<MAX_VENTAS Entonces
                totalVentas <- totalVentas+1
                Escribir ''
                Escribir ' -- Registrar Venta --'
                Escribir ' Producto : 'Sin Saltar
                Leer producto[totalVentas]
                // Validación de la cantidad
                Repetir
                    Escribir ' Cantidad : 'Sin Saltar
                    Leer cantidad[totalVentas]
                    Si cantidad<=0 Entonces
                        Escribir ' ERROR La cantidad
debe ser mayor a 0.'
                    FinSi
                Hasta Que cantidad>0
                // Validación del precio
                Repetir
                    Escribir ' Precio u. : 'Sin Saltar
                    Leer precio[totalVentas]
                    Si precio<=0 Entonces
                        Escribir ' ERROR El precio debe
ser mayor a 0.'
                    FinSi
                Hasta Que precio>0
                // Simulación del método calcularTotal()
                total[totalVentas] <-
cantidad[totalVentas]*precio[totalVentas]
                // Simulación del método mostrarFactura()
                Escribir ''
                Escribir ' ===== FACTURA GENERADA
===== '
                Escribir ' -----'
                Escribir ' Producto : ',
                Escribir ' Cantidad : ',
                Escribir ' Precio u. : $',
                Escribir ' TOTAL : $',
                Escribir ' -----'
                Escribir ' -----'
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



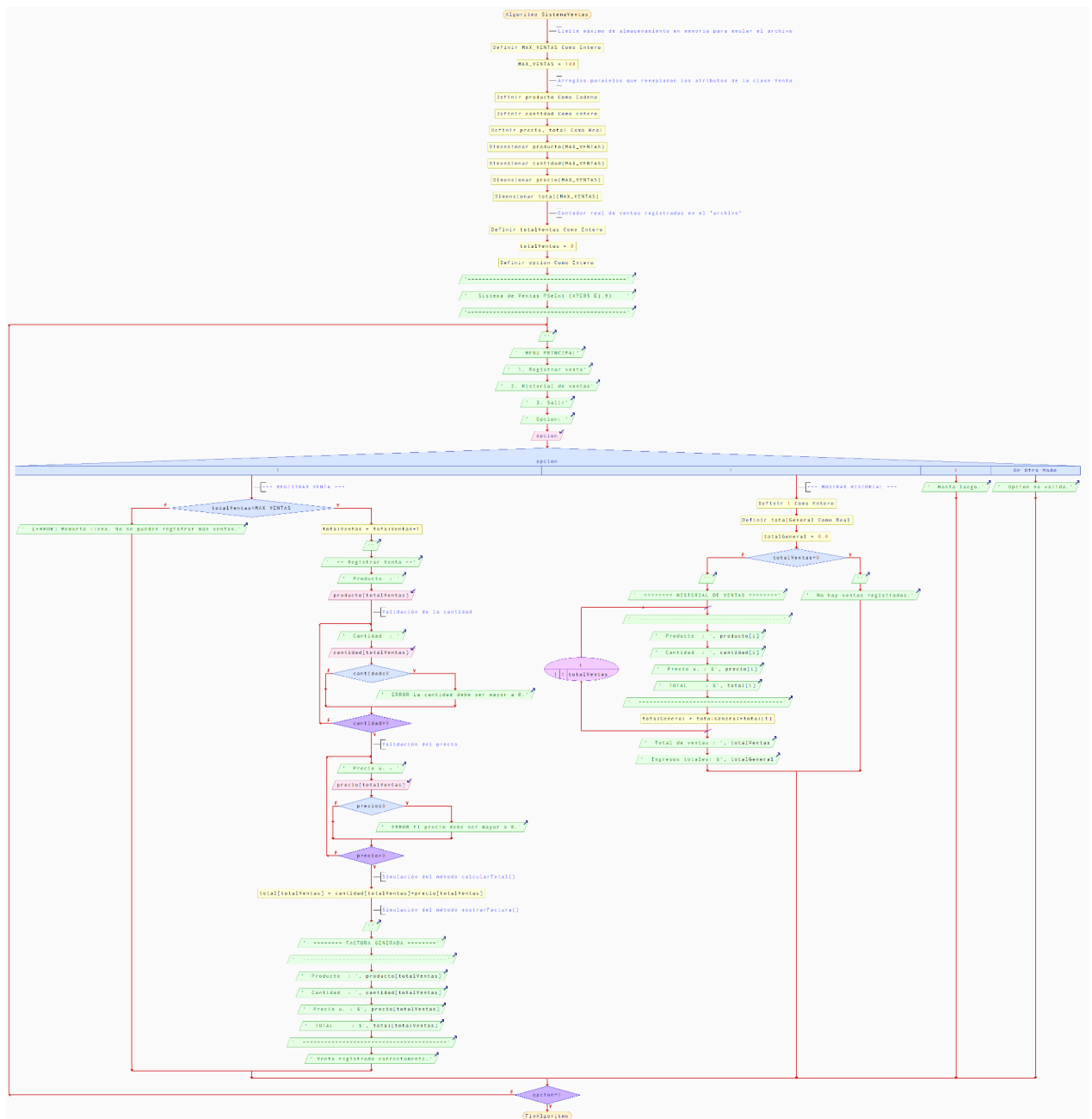
```

                                Escribir ' Venta registrada correctamente.'
                                SiNo
                                Escribir ' [ERROR] Memoria llena. No se
pueden registrar mas ventas.'
                                FinSi
2:
    // --- MOSTRAR HISTORIAL ---
    Definir i Como Entero
    Definir totalGeneral Como Real
    totalGeneral <- 0.0
    Si totalVentas=0 Entonces
        Escribir ''
        Escribir ' No hay ventas registradas.'
    SiNo
        Escribir ''
        Escribir ' ===== HISTORIAL DE VENTAS
===== '
                                Para i<-1 Hasta totalVentas Con Paso 1 Hacer
                                Escribir ' -----
                                Escribir ' Producto : ', producto[i]
                                Escribir ' Cantidad : ', cantidad[i]
                                Escribir ' Precio u. : $', precio[i]
                                Escribir ' TOTAL : $', total[i]
                                Escribir ' -----
                                totalGeneral <- totalGeneral+total[i]
                                FinPara
                                Escribir ' Total de ventas : ', totalVentas
                                Escribir ' Ingresos totales: $',
totalGeneral
                                FinSi
3:
    Escribir ' Hasta luego.'
De Otro Modo:
    Escribir ' Opcion no valida.'
FinSegún
Hasta Que opcion=3
FinAlgoritmo
```

5. Diagrama de flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



6. Codificación C++

```
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <iomanip>
using namespace std;

// Archivo donde se guardan las ventas
const string ARCHIVO = "ventas.txt";

// Clase Venta
class Venta {
private:
    string producto;
    int cantidad;
    double precio;
    double total;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
public:
    // Constructor por defecto
    Venta() : producto(""), cantidad(0), precio(0.0), total(0.0) {}

    // Constructor con parámetros
    Venta(string _producto, int _cantidad, double _precio) {
        producto = _producto;
        cantidad = _cantidad;
        precio = _precio;
        calcularTotal();
    }

    // Métodos GET
    string getProducto() const { return producto; }
    int getCantidad() const { return cantidad; }
    double getPrecio() const { return precio; }
    double getTotal() const { return total; }

    // Métodos SET
    void setProducto(string _producto) { producto = _producto; }
    void setCantidad(int _cantidad) { cantidad = _cantidad; }
    void setPrecio(double _precio) { precio = _precio; }

    // Calcular total de la venta
    void calcularTotal() {
        total = cantidad * precio;
    }

    // Mostrar factura de la venta
    void mostrarFactura() const {
        cout << fixed << setprecision(2);
        cout << " -----" << endl;
        cout << " Producto : " << producto << endl;
        cout << " Cantidad : " << cantidad << endl;
        cout << " Precio u. : $" << precio << endl;
        cout << " TOTAL : $" << total << endl;
        cout << " -----" << endl;
    }
};

// Función: registrarVenta
// Solicita datos y guarda la venta en el archivo
void registrarVenta() {
    string producto;
    int cantidad;
    double precio;

    cout << "\n -- Registrar Venta --" << endl;
    cout << " Producto : "; cin >> producto;

    do {
        cout << " Cantidad : "; cin >> cantidad;
        if (cantidad <= 0)
            cout << " ERROR La cantidad debe ser mayor a 0." << endl;
    } while (cantidad <= 0);

    do {
        cout << " Precio u. : "; cin >> precio;
        if (precio <= 0)
            cout << " ERROR El precio debe ser mayor a 0." << endl;
    } while (precio <= 0);

    Venta v(producto, cantidad, precio);

    // Mostrar la factura generada
    cout << "\n ===== FACTURA GENERADA =====" << endl;
    v.mostrarFactura();
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Guardar en archivo (modo append)
ofstream archivo(ARCHIVO, ios::app);
if (!archivo.is_open()) {
    cout << " ERROR No se pudo abrir el archivo de ventas." << endl;
    return;
}

archivo << fixed << setprecision(2);
archivo << v.getProducto() << "|"
        << v.getCantidad() << "|"
        << v.getPrecio() << "|"
        << v.getTotal() << endl;

archivo.close();
cout << " Venta registrada correctamente." << endl;
}

// Función: mostrarHistorial
// Lee el archivo y muestra todas las ventas registradas
void mostrarHistorial() {
    ifstream archivo(ARCHIVO);
    if (!archivo.is_open()) {
        cout << " No hay ventas registradas." << endl;
        return;
    }

    cout << "\n ===== HISTORIAL DE VENTAS =====" << endl;

    string linea;
    int contador = 0;
    double totalGeneral = 0.0;

    while (getline(archivo, linea)) {
        if (linea.empty()) continue;

        stringstream ss(linea);
        string producto, sCantidad, sPrecio, sTotal;
        getline(ss, producto, '|');
        getline(ss, sCantidad, '|');
        getline(ss, sPrecio, '|');
        getline(ss, sTotal, '|');

        int cantidad = stoi(sCantidad);
        double precio = stod(sPrecio);
        double total = stod(sTotal);

        Venta v(producto, cantidad, precio);
        v.mostrarFactura();

        totalGeneral += total;
        contador++;
    }

    archivo.close();

    if (contador == 0) {
        cout << " No hay ventas registradas." << endl;
    } else {
        cout << fixed << setprecision(2);
        cout << " Total de ventas : " << contador << endl;
        cout << " Ingresos totales: $" << totalGeneral << endl;
    }
}

// Función principal - Menú interactivo
int main() {
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
int opcion;

cout << "===== " << endl;
cout << "    Sistema de Ventas C++ (APE05 Ej.9)    " << endl;
cout << "===== " << endl;

do {
    cout << "\n MENU PRINCIPAL" << endl;
    cout << "  1. Registrar venta" << endl;
    cout << "  2. Historial de ventas" << endl;
    cout << "  3. Salir" << endl;
    cout << "  Opcion: ";
    cin  >> opcion;

    switch (opcion) {
        case 1: registrarVenta(); break;
        case 2: mostrarHistorial(); break;
        case 3: cout << "  Hasta luego." << endl; break;
        default: cout << "  Opcion no valida." << endl;
    }

} while (opcion != 3);

return 0;
}
```

7. Codificación Java

```
import java.io.*;
import java.util.Scanner;

public class Ej9main {

    // Archivo donde se guardan las ventas
    private static final String ARCHIVO = "ventas.txt";
    private static Scanner sc = new Scanner(System.in);

    // Método: registrarVenta
    // Solicita datos, genera factura y guarda en archivo
    public static void registrarVenta() throws IOException {
        System.out.println("\n  -- Registrar Venta --");

        System.out.print("  Producto  : ");
        String producto = sc.next();

        int cantidad;
        do {
            System.out.print("  Cantidad  : ");
            cantidad = sc.nextInt();
            if (cantidad <= 0)
                System.out.println("  La cantidad debe ser mayor a 0.");
        } while (cantidad <= 0);

        double precio;
        do {
            System.out.print("  Precio u.  : ");
            precio = sc.nextDouble();
            if (precio <= 0)
                System.out.println("  El precio debe ser mayor a 0.");
        } while (precio <= 0);

        Ej9Venta v = new Ej9Venta(producto, cantidad, precio);

        // Mostrar factura generada
        System.out.println("\n  ===== FACTURA GENERADA =====");
        v.mostrarFactura();
    }
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// Guardar en archivo (modo append)
FileWriter fw = new FileWriter(ARCHIVO, true);
BufferedWriter bw = new BufferedWriter(fw);
bw.write(v.toArchivo());
bw.newLine();
bw.close();
fw.close();

System.out.println(" Venta registrada correctamente.");
}

// Método: mostrarHistorial
// Lee el archivo y muestra todas las ventas
public static void mostrarHistorial() throws IOException {
File f = new File(ARCHIVO);
if (!f.exists()) {
System.out.println(" No hay ventas registradas.");
return;
}

System.out.println("\n ===== HISTORIAL DE VENTAS =====");

BufferedReader br = new BufferedReader(new FileReader(ARCHIVO));
String linea;
int contador = 0;
double totalGeneral = 0.0;

while ((linea = br.readLine()) != null) {
if (linea.trim().isEmpty()) continue;
Ej9Venta v = Ej9Venta.fromArchivo(linea);
v.mostrarFactura();
totalGeneral += v.getTotal();
contador++;
}
br.close();

if (contador == 0) {
System.out.println(" [INFO] No hay ventas registradas.");
} else {
System.out.printf(" Total de ventas : %d\n", contador);
System.out.printf(" Ingresos totales : $%.2f\n", totalGeneral);
}
}

// Método principal - Menú interactivo
public static void main(String[] args) {
System.out.println("=====");
System.out.println(" Sistema de Ventas Java (APE05 Ej.9) ");
System.out.println("=====");

int opcion;
do {
System.out.println("\n MENU PRINCIPAL");
System.out.println(" 1. Registrar venta");
System.out.println(" 2. Historial de ventas");
System.out.println(" 3. Salir");
System.out.print(" Opcion: ");
opcion = sc.nextInt();

try {
switch (opcion) {
case 1: registrarVenta(); break;
case 2: mostrarHistorial(); break;
case 3: System.out.println(" Hasta luego."); break;
default: System.out.println(" Opcion no valida.");
}
}
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
} catch (IOException e) {  
System.out.println(" Problema con el archivo: " + e.getMessage());  
}  
  
} while (opcion != 3);  
  
sc.close();  
}  
}
```

8. Prueba Solución C++

```
===== HISTORIAL DE VENTAS =====  
-----  
Producto : Papas  
Cantidad : 2  
Precio u. : $1.00  
TOTAL : $2.00  
-----  
Producto : Refresco  
Cantidad : 2  
Precio u. : $0.50  
TOTAL : $1.00  
-----  
Total de ventas : 2  
Ingresos totales: $3.00  
  
MENU PRINCIPAL  
1. Registrar venta  
2. Historial de ventas  
3. Salir  
Opcion: 3  
Hasta luego.  
PS C:\Users\Matias\Desktop\deberes uta\alg promg log\c++>
```

```
PS C:\Users\Matias\Desktop\deberes uta\alg promg log\c++>  
main }  
=====  
Sistema de Ventas C++ (APE05 Ej.9)  
=====  
  
MENU PRINCIPAL  
1. Registrar venta  
2. Historial de ventas  
3. Salir  
Opcion: 1  
  
-- Registrar Venta --  
Producto : Papas  
Cantidad : 2  
Precio u. : 1  
  
===== FACTURA GENERADA =====  
-----  
Producto : Papas  
Cantidad : 2  
Precio u. : $1.00  
TOTAL : $2.00  
-----  
Venta registrada correctamente.  
  
MENU PRINCIPAL  
1. Registrar venta  
2. Historial de ventas  
3. Salir  
Opcion: 1  
  
-- Registrar Venta --  
Producto : Refresco  
Cantidad : 2  
Precio u. : 0.50  
  
===== FACTURA GENERADA =====  
-----  
Producto : Refresco  
Cantidad : 2  
Precio u. : $0.50  
TOTAL : $1.00  
-----  
Venta registrada correctamente.  
  
MENU PRINCIPAL  
1. Registrar venta  
2. Historial de ventas  
3. Salir  
Opcion: 2
```

9. Prueba de escritorio Java



```
===== HISTORIAL DE VENTAS =====
-----
Producto : Chifles
Cantidad : 3
Precio u. : $1,00
TOTAL    : $3,00
-----
Producto : Queso
Cantidad : 1
Precio u. : $4,00
TOTAL    : $4,00
-----
Total de ventas : 2
Ingresos totales : $7,00

MENU PRINCIPAL
1. Registrar venta
2. Historial de ventas
3. Salir
Opcion: 3
Hasta luego.
PS C:\Users\Matias\Desktop\deberes uta\alg promg log\Java>
```

```
PS C:\Users\Matias\Desktop\deberes uta\alg promg log\Java> cd
n }

=====
Sistema de Ventas Java (APE05 Ej.9)
=====

MENU PRINCIPAL
1. Registrar venta
2. Historial de ventas
3. Salir
Opcion: 1

-- Registrar Venta --
Producto : Chifles
Cantidad : 3
Precio u. : 1

===== FACTURA GENERADA =====
-----
Producto : Chifles
Cantidad : 3
Precio u. : $1,00
TOTAL    : $3,00
-----
Venta registrada correctamente.

MENU PRINCIPAL
1. Registrar venta
2. Historial de ventas
3. Salir
Opcion: 1

-- Registrar Venta --
Producto : Queso
Cantidad : 1
Precio u. : 4

===== FACTURA GENERADA =====
-----
Producto : Queso
Cantidad : 1
Precio u. : $4,00
TOTAL    : $4,00
-----
Venta registrada correctamente.

MENU PRINCIPAL
1. Registrar venta
2. Historial de ventas
3. Salir
Opcion: 2
```

Ejercicio 10 – Mini Proyecto Integrador: Sistema de Gestión de Productos

1. Análisis

El Ejercicio 10 constituye el Mini Proyecto Integrador de la APE 05. Se implementa un sistema de gestión de productos con persistencia en archivo de texto, que cubre las cuatro operaciones CRUD completas:

- **CREATE (Agregar):** registra un nuevo producto validando que el código no esté duplicado y que los valores numéricos sean positivos.
- **READ (Buscar/Listar):** permite buscar por código o nombre con comparación sin distinción de mayúsculas, y listar el catálogo completo.
- **UPDATE (Modificar):** localiza un producto por código y actualiza solo los campos que el usuario desea cambiar (campos vacíos conservan el valor anterior).
- **DELETE (Eliminar):** solicita confirmación antes de borrar el registro del producto de la lista y del archivo.

El archivo "productos.txt" almacena cada producto en una línea con formato código|nombre|categoría|precio|stock. El programa carga los registros al inicio y guarda inmediatamente tras cada operación de escritura, garantizando persistencia y coherencia. Se aplica POO completa con la clase Producto, encapsulamiento mediante atributos privados (Java) o públicos encapsulados (C++) y métodos separados por responsabilidad.



2. Declaración de Variables

ARCHIVO	String (const)	Nombre del archivo de persistencia "productos.txt"
codigo	String	Código único identificador del producto
nombre	String	Nombre descriptivo del producto
categoria	String	Categoría o tipo al que pertenece el producto
precio	double / Real	Precio unitario del producto en dólares
stock	int / Entero	Cantidad de unidades disponibles en inventario
opcion	int / Entero	Opción seleccionada por el usuario en el menú
busqueda	String	Criterio de búsqueda ingresado por el usuario
conf	char / String	Confirmación del usuario antes de eliminar (s/n)
lista	vector / List	Colección principal de todos los objetos Producto

3. Algoritmo paso a paso

1. Abrir "productos.txt" y cargar todos los productos existentes en la lista.
2. Mostrar el menú: 1-Agregar, 2-Buscar, 3-Modificar, 4-Eliminar, 5-Listar, 0-Salir.
3. Leer la opción del usuario.
4. Opción 1 (Agregar): leer código, validar que no exista; leer nombre, categoría, precio y stock; validar valores positivos; añadir a la lista y guardar en el archivo.
5. Opción 2 (Buscar): leer criterio; recorrer la lista comparando (sin distinción de mayúsculas) con código y nombre; mostrar los coincidentes o mensaje de no encontrado.
6. Opción 3 (Modificar): leer código a modificar; buscar en la lista; para cada campo, si el usuario ingresa valor nuevo se actualiza, si no se mantiene el anterior; guardar en archivo.
7. Opción 4 (Eliminar): leer código; buscar en la lista; pedir confirmación; si se confirma, eliminar de la lista y guardar; si no, cancelar.
8. Opción 5 (Listar): mostrar todos los productos numerados con todos sus campos y el total al final.
9. Opción 0: terminar el programa.
10. Para cualquier otra opción: mostrar mensaje de opción inválida.
11. Repetir desde el paso 2 hasta que la opción sea 0.

4.Pseudocodigo

Proceso SistemaProductos

Definir codigos, nombres, categorias Como Cadena

Dimension codigos[100]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
Dimension nombres[100]

Dimension categorias[100]

Definir precios Como Real

Dimension precios[100]


Definir stocks Como Entero

Dimension stocks[100]

Definir opcion, i, total, stock Como Entero

Definir precio Como Real

Definir codigo, nombre, categoria, buscar Como Cadena

Definir encontrado Como Logico

total <- 0

Repetir

    Escribir "1. Agregar"

    Escribir "2. Buscar"

    Escribir "3. Listar"

    Escribir "0. Salir"

    Leer opcion

    Segun opcion Hacer

        1:

            Escribir "Codigo:"

            Leer codigo

            encontrado <- Falso

            Para i <- 0 Hasta total - 1 Hacer

                Si codigos[i] = codigo Entonces

                    encontrado <- Verdadero

                FinSi

            FinPara

            Si encontrado = Verdadero Entonces

                Escribir "Codigo ya existe"

            SiNo

                Escribir "Nombre:"

                Leer nombre
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```

    Escribir "Categoria:"

    Leer categoria

    Escribir "Precio:"

    Leer precio

    Escribir "Stock:"

    Leer stock

    codigos[total] <- codigo
    nombres[total] <- nombre

    categorias[total] <- categoria

    precios[total] <- precio

    stocks[total] <- stock

    total <- total + 1

    Escribir "Producto agregado"

FinSi

2:

    Escribir "Codigo a buscar:"

    Leer buscar

    encontrado <- Falso

    Para i <- 0 Hasta total - 1 Hacer

        Si codigos[i] = buscar Entonces

            Escribir "Codigo: ", codigos[i]

            Escribir "Nombre: ", nombres[i]

            Escribir "Categoria: ", categorias[i]

            Escribir "Precio: ", precios[i]

            Escribir "Stock: ", stocks[i]

            encontrado <- Verdadero

        FinSi

    FinPara

    Si encontrado = Falso Entonces

        Escribir "Producto no encontrado"

    FinSi

3:

    Para i <- 0 Hasta total - 1 Hacer

        Escribir "-----"
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
    Escribir "Codigo: ", codigos[i]

    Escribir "Nombre: ", nombres[i]

    Escribir "Categoria: ", categorias[i]

    Escribir "Precio: ", precios[i]

    Escribir "Stock: ", stocks[i]

FinPara

0:

    Escribir "Fin del programa"

De Otro Modo:

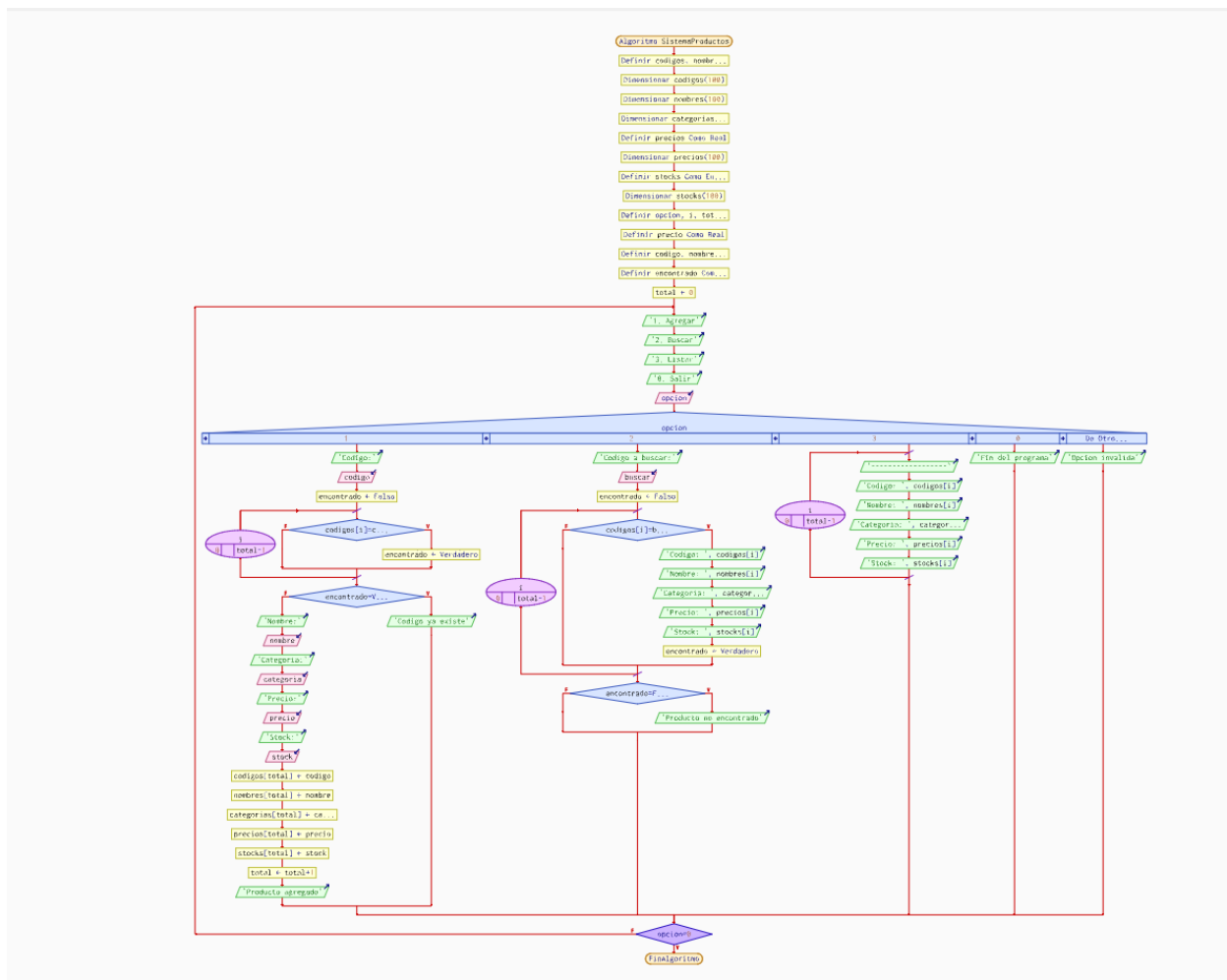
    Escribir "Opcion invalida"

FinSegun

Hasta Que opcion = 0

FinProceso}
```

5. Diagrama de Flujo



6. Código en C++



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
// =====  
  
// APE 05 - Ejercicio 10: Mini Proyecto Integrador  
  
// Sistema de Gestion de Productos (CRUD)  
  
// Asignatura: Algoritmos y Logica de Programacion  
  
// Docente: Jose Ruben Caiza  
  
// =====  
  
#include <iostream>  
  
#include <fstream>  
  
#include <string>  
  
#include <vector>  
  
#include <sstream>  
  
#include <iomanip>  
  
#include <algorithm>  
  
using namespace std;  
  
  
// Clase Producto - encapsula los atributos y comportamiento de cada producto  
class Producto {  
public:  
    string codigo, nombre, categoria;  
    double precio;  
    int    stock;  
  
    Producto(string c, string n, string cat, double p, int s)  
        : codigo(c), nombre(n), categoria(cat), precio(p), stock(s) {}  
  
    void mostrar() const {  
        cout << "  Codigo      : " << codigo      << endl;  
        cout << "  Nombre       : " << nombre       << endl;  
        cout << "  Categoria   : " << categoria << endl;  
        cout << "  Precio      : $" << fixed << setprecision(2) << precio <<  
endl;  
        cout << "  Stock       : " << stock << " unidades" << endl;  
    }  
}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
string toLinea() const {

    stringstream ss;

    ss << fixed << setprecision(2) << precio;

    return codigo + "|" + nombre + "|" + categoria + "|" + ss.str() + "|"
+ to_string(stock);

}

};

// Metodo auxiliar: divide una linea por delimitador
vector<string> dividir(const string& linea, char delim) {

    vector<string> partes; stringstream ss(linea); string tok;

    while (getline(ss, tok, delim)) partes.push_back(tok);

    return partes;

}

// Metodo: cargar productos desde el archivo
vector<Producto> cargarProductos(const string& archivo) {

    vector<Producto> lista; ifstream f(archivo);

    if (!f.is_open()) return lista;

    string linea;

    while (getline(f, linea)) {

        if (!linea.empty()) {

            auto d = dividir(linea, '|');

            if (d.size() == 5)

                lista.push_back(Producto(d[0], d[1], d[2], stod(d[3]),
stoi(d[4])));

        }

    }

    f.close(); return lista;

}

// Metodo: guardar todos los productos en el archivo
void guardarProductos(const string& archivo, const vector<Producto>& lista) {

    ofstream f(archivo);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
for (const auto& p : lista) f << p.toLinea() << "\n";
}

// CRUD 1 - CREATE: agregar nuevo producto
void agregarProducto(const string& archivo, vector<Producto>& lista) {
    string cod, nom, cat; double precio; int stock;
    cout << "\n--- Agregar Producto ---\n";
    cout << "Codigo      : "; cin >> cod; cin.ignore();
    // Validar codigo unico
    for (const auto& p : lista)
        if (p.codigo == cod) { cout << "Error: el codigo ya existe.\n";
return; }

    cout << "Nombre      : "; getline(cin, nom);
    cout << "Categoria : "; getline(cin, cat);
    cout << "Precio     : "; cin >> precio;
    cout << "Stock      : "; cin >> stock;

    if (precio < 0 || stock < 0) { cout << "Error: precio y stock deben ser
positivos.\n"; return; }

    lista.push_back(Producto(cod, nom, cat, precio, stock));
    guardarProductos(archivo, lista);
    cout << "Producto agregado correctamente.\n";
}

// CRUD 2 - READ: buscar producto
void buscarProducto(const vector<Producto>& lista) {
    cin.ignore();
    cout << "\nCriterio de busqueda: "; string b; getline(cin, b);
    string bl = b; transform(bl.begin(), bl.end(), bl.begin(), ::tolower);
    bool ok = false;
    for (const auto& p : lista) {
        string cl = p.codigo, nl = p.nombre;
        transform(cl.begin(), cl.end(), cl.begin(), ::tolower);
        transform(nl.begin(), nl.end(), nl.begin(), ::tolower);
        if (cl.find(bl) != string::npos || nl.find(bl) != string::npos) {
            p.mostrar(); cout << "-----\n"; ok =
true;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
}  
  
}  
  
if (!ok) cout << "Sin resultados para: " << b << "\n";  
  
}  
  
  
// CRUD 3 - UPDATE: modificar producto  
  
void modificarProducto(const string& archivo, vector<Producto>& lista) {  
  
    cin.ignore();  
  
    cout << "\n--- Modificar Producto ---\nCodigo a modificar: "; string cod;  
    getline(cin, cod);  
  
    for (auto& p : lista) {  
  
        if (p.codigo == cod) {  
  
            string nuevo; double np; int ns;  
  
            cout << "Nuevo nombre      (Enter=mantener): "; getline(cin, nuevo);  
            if (!nuevo.empty()) p.nombre = nuevo;  
  
            cout << "Nueva categoria (Enter=mantener): "; getline(cin, nuevo);  
            if (!nuevo.empty()) p.categoria = nuevo;  
  
            cout << "Nuevo precio      (0=mantener): "; cin >> np; if (np > 0)  
p.precio = np;  
  
            cout << "Nuevo stock      (-1=mantener): "; cin >> ns; if (ns >= 0)  
p.stock = ns;  
  
            guardarProductos(archivo, lista);  
  
            cout << "Producto modificado correctamente.\n"; return;  
  
        }  
  
    }  
  
    cout << "Producto con codigo '" << cod << "' no encontrado.\n";  
  
}  
  
  
// CRUD 4 - DELETE: eliminar producto  
  
void eliminarProducto(const string& archivo, vector<Producto>& lista) {  
  
    cin.ignore();  
  
    cout << "\n--- Eliminar Producto ---\nCodigo a eliminar: "; string cod;  
    getline(cin, cod);  
  
    for (int i = 0; i < (int)lista.size(); i++) {  
  
        if (lista[i].codigo == cod) {  
  
            cout << "Confirmar eliminacion de '" << lista[i].nombre << "'?  
(s/n): ";
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
char conf; cin >> conf;

if (conf == 's' || conf == 'S') {

    lista.erase(lista.begin() + i);

    guardarProductos(archivo, lista);

    cout << "Producto eliminado correctamente.\n";

} else cout << "Eliminacion cancelada.\n";

return;

}

}

cout << "Producto no encontrado.\n";

}

// CRUD 2 - READ: listar todos

void listarTodos(const vector<Producto>& lista) {

    if (lista.empty()) { cout << "No hay productos registrados.\n"; return; }

    cout << "\n===== TODOS LOS PRODUCTOS =====\n";

    for (int i = 0; i < (int)lista.size(); i++) {

        cout << "Producto #" << (i+1) << "\n";

        lista[i].mostrar();

        cout << "---\n";

    }

    cout << "Total: " << lista.size() << " producto(s).\n";

}

int main() {

    const string archivo = "productos.txt";

    vector<Producto> lista = cargarProductos(archivo);

    int op;

    do {

        cout << "\n===== SISTEMA DE PRODUCTOS =====\n";

        cout << "1. Agregar producto\n2. Buscar producto\n";

        cout << "3. Modificar producto\n4. Eliminar producto\n";

        cout << "5. Listar todos\n0. Salir\nOpcion: "; cin >> op;

        switch (op) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
case 1: agregarProducto(archivo, lista);    break;

case 2: buscarProducto(lista);              break;

case 3: modificarProducto(archivo, lista); break;

case 4: eliminarProducto(archivo, lista);  break;

case 5: listarTodos(lista);                 break;

case 0: cout << "Saliendo del sistema...\n"; break;

default: cout << "Opcion invalida.\n";

    }

} while (op != 0);

return 0;

}
```

7. Prueba de escritorio

```
===== SISTEMA DE PRODUCTOS =====
1. Agregar producto
2. Buscar producto
3. Modificar producto
4. Eliminar producto
5. Listar todos
0. Salir
Opcion: 1

--- Agregar Producto ---
Codigo    : 2345
Nombre    : Avena
Categoria : cereales
Precio    : 3
Stock     : 5
Producto agregado correctamente.

===== SISTEMA DE PRODUCTOS =====
1. Agregar producto
2. Buscar producto
3. Modificar producto
4. Eliminar producto
5. Listar todos
0. Salir
Opcion: 1

--- Agregar Producto ---
Codigo    : 2345
Error: el codigo ya existe.

===== SISTEMA DE PRODUCTOS =====
1. Agregar producto
2. Buscar producto
3. Modificar producto
4. Eliminar producto
5. Listar todos
0. Salir
Opcion: 1

--- Agregar Producto ---
Codigo    : 6789
Nombre    : Yogurt
Categoria : Lacteos
Precio    : 6
Stock     : 9
Producto agregado correctamente.
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
===== SISTEMA DE PRODUCTOS =====
1. Agregar producto
2. Buscar producto
3. Modificar producto
4. Eliminar producto
5. Listar todos
0. Salir
Opcion: 5

===== TODOS LOS PRODUCTOS =====
Producto #1
Codigo      : 2345
Nombre      : Avena
Categoria   : cereales
Precio      : $3.00
Stock       : 5 unidades
---
Producto #2
Codigo      : 6789
Nombre      : Yogurt
Categoria   : Lacteos
Precio      : $6.00
Stock       : 9 unidades
---
Total: 2 producto(s).

===== SISTEMA DE PRODUCTOS =====
1. Agregar producto
2. Buscar producto
3. Modificar producto
4. Eliminar producto
5. Listar todos
0. Salir
Opcion: 0
Saliendo del sistema...
```

8.Codigo en JAVA

Clase: productos.java Metodos,Objetos,Atributos

```
package com.mycompany.productos;

public class Productos {

    private String codigo;

    private String nombre;

    private String categoria;

    private double precio;

    private int stock;

    public Productos(String codigo, String nombre, String categoria, double
precio, int stock) {

        this.codigo = codigo;

        this.nombre = nombre;

        this.categoria = categoria;

        this.precio = precio;

        this.stock = stock;

    }

    public String getCodigo() {

        return codigo;

    }

    public String getNombre() {

        return nombre;

    }

    public String getCategoria() {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        return categoria;
    }

    public double getPrecio() {
        return precio;
    }

    public int getStock() {
        return stock;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }

    public void mostrar() {
        System.out.println("Codigo: " + codigo);
        System.out.println("Nombre: " + nombre);
        System.out.println("Categoria: " + categoria);
        System.out.println("Precio: " + precio);
        System.out.println("Stock: " + stock);
    }
}
```

CLASE PRINICIPAL MAIN

```
package com.mycompany.productos;

import java.io.*;

import java.util.*;

public class mainproductos {

    static final String ARCHIVO = "productos.txt";
```




UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
static ArrayList<Productos> lista = new ArrayList<>();

static Scanner sc = new Scanner(System.in);

// ===== CARGAR =====

public static void cargarProductos() {

    lista.clear();

    try {

        BufferedReader br = new BufferedReader(new FileReader(ARCHIVO));

        String linea;

        while ((linea = br.readLine()) != null) {

            String d[] = linea.split("[|]");

            if (d.length == 5) {

                Productos p = new Productos(

                    d[0],

                    d[1],

                    d[2],

                    Double.parseDouble(d[3]),

                    Integer.parseInt(d[4])

                );

                lista.add(p);

            }

        }

        br.close();

    } catch (Exception e) {

        System.out.println("Archivo no encontrado.");

    }

}

// ===== GUARDAR =====

public static void guardarProductos() {

    try {

        BufferedWriter bw = new BufferedWriter(new FileWriter(ARCHIVO));

        for (Productos p : lista) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        bw.write(

            p.getCodigo() + "|" +

            p.getNombre() + "|" +

            p.getCategoria() + "|" +

            p.getPrecio() + "|" +

            p.getStock()

        );

        bw.newLine();

    }

    bw.close();

} catch (Exception e) {

    System.out.println("Error al guardar.");

}

}

// ===== AGREGAR =====

public static void agregarProducto() {

    System.out.println("\n--- AGREGAR PRODUCTO ---");

    System.out.print("Codigo: ");

    String cod = sc.nextLine();

    for (Productos p : lista) {

        if (p.getCodigo().equals(cod)) {

            System.out.println("Codigo ya existe.");

            return;

        }

    }

    System.out.print("Nombre: ");

    String nom = sc.nextLine();

    System.out.print("Categoria: ");

    String cat = sc.nextLine();

    System.out.print("Precio: ");

    double precio = Double.parseDouble(sc.nextLine());
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.print("Stock: ");

int stock = Integer.parseInt(sc.nextLine());

Productos nuevo = new Productos(cod, nom, cat, precio, stock);
lista.add(nuevo);

guardarProductos();

System.out.println("Producto agregado.");
}

// ===== BUSCAR =====

public static void buscarProducto() {

    System.out.print("\nBuscar: ");

    String b = sc.nextLine().toLowerCase();

    boolean encontrado = false;

    for (Productos p : lista) {

        if (p.getCodigo().toLowerCase().contains(b)
            || p.getNombre().toLowerCase().contains(b)) {

            p.mostrar();

            System.out.println("-----");

            encontrado = true;

        }

    }

    if (!encontrado) {

        System.out.println("Producto no encontrado.");

    }

}

// ===== MODIFICAR =====

public static void modificarProducto() {

    System.out.print("\nCodigo a modificar: ");

    String cod = sc.nextLine();

    for (Productos p : lista) {

        if (p.getCodigo().equals(cod)) {

            System.out.print("Nuevo nombre: ");
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
p.setNombre(sc.nextLine());

System.out.print("Nueva categoria: ");

p.setCategoria(sc.nextLine());

System.out.print("Nuevo precio: ");

p.setPrecio(Double.parseDouble(sc.nextLine()));

System.out.print("Nuevo stock: ");

p.setStock(Integer.parseInt(sc.nextLine()));

guardarProductos();

System.out.println("Producto modificado.");

return;

}

}

System.out.println("Producto no encontrado.");

}

// ===== ELIMINAR =====

public static void eliminarProducto() {

    System.out.print("\nCodigo a eliminar: ");

    String cod = sc.nextLine();

    for (int i = 0; i < lista.size(); i++) {

        if (lista.get(i).getCodigo().equals(cod)) {

            lista.remove(i);

            guardarProductos();

            System.out.println("Producto eliminado.");

            return;

        }

    }

    System.out.println("Producto no encontrado.");

}

// ===== LISTAR =====

public static void listarProductos() {

    if (lista.isEmpty()) {

        System.out.println("No hay productos.");

        return;

    }

}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
System.out.println("\n===== LISTA PRODUCTOS =====");

for (Productos p : lista) {

    p.mostrar();

    System.out.println("-----");

}

}

// ===== MAIN =====

public static void main(String[] args) {

    cargarProductos();

    int op;

    do {

        System.out.println("\n===== SISTEMA CRUD =====");

        System.out.println("1. Agregar");

        System.out.println("2. Buscar");

        System.out.println("3. Modificar");

        System.out.println("4. Eliminar");

        System.out.println("5. Listar");

        System.out.println("0. Salir");

        System.out.print("Opcion: ");

        op = Integer.parseInt(sc.nextLine());

        switch (op) {

            case 1:

                agregarProducto();

                break;

            case 2:

                buscarProducto();

                break;

            case 3:

                modificarProducto();

                break;

            case 4:
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        eliminarProducto();  
  
        break;  
  
    case 5:  
  
        listarProductos();  
  
        break;  
  
    case 0:  
  
        System.out.println("Saliendo...");  
  
        break;  
  
    default:  
  
        System.out.println("Opcion invalida.");  
  
    }  
  
    } while (op != 0);  
  
    sc.close();  
  
    }  
  
    }
```

9.Pruebas de escritorio JAVA



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



===== SISTEMA CRUD =====

1. Agregar
2. Buscar
3. Modificar
4. Eliminar
5. Listar
0. Salir

Opcion: 1

--- AGREGAR PRODUCTO ---

Codigo: 3456
Nombre: Pan
Categoria: Cereales
Precio: 8
Stock: 6
Producto agregado.

===== SISTEMA CRUD =====

1. Agregar
2. Buscar
3. Modificar
4. Eliminar
5. Listar
0. Salir

Opcion: 1

--- AGREGAR PRODUCTO ---

Codigo: 876
Nombre: Pollo
Categoria: Proteina
Precio: 5
Stock: 4
Producto agregado.

===== SISTEMA CRUD =====

1. Agregar
2. Buscar
3. Modificar
4. Eliminar
5. Listar
0. Salir

Opcion: 0

===== SISTEMA CRUD =====

1. Agregar
2. Buscar
3. Modificar
4. Eliminar
5. Listar
0. Salir

Opcion: 0

Saliendo...

BUILD SUCCESS

Total time: 01:15 min
Finished at: 2026-05-22T15:19:35-05:00

.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☐ Responsabilidad

.9 Conclusiones

- La manipulación de ficheros con ofstream e ifstream en C++ permite persistir datos entre ejecuciones sin necesidad de bases de datos, siendo suficiente para sistemas de información de pequeña y mediana escala.
- El uso de ios::app en la apertura de archivos de escritura garantiza que los registros anteriores no sean sobre escritos, aspecto fundamental para sistemas de registro acumulativo como la agenda y el control de asistencia.



- La separación de responsabilidades mediante clases y métodos (guardar, leer, mostrar, calcular) facilita el mantenimiento del código y reduce la duplicación, aplicando el principio de responsabilidad única de la POO.

.10 Recomendaciones

- Validar siempre los datos de entrada en los programas que manejan archivos, especialmente tipos numéricos, ya que distintos sistemas operativos pueden usar diferente formato decimal (punto o coma), lo que puede causar errores en tiempo de ejecución.
- Utilizar el modo append (`ios::app` en C++ y `FileWriter(true)` en Java) al escribir en archivos, para garantizar que los registros anteriores no se pierdan cada vez que el programa se ejecuta.
- Mantener una estructura clara del repositorio en GitHub separando los archivos por ejercicio y lenguaje, realizando commits progresivos con mensajes descriptivos que reflejen el avance del trabajo.

.1 Referencias bibliográficas

- [1] R. S. Pressman y B. R. Maxim, *Ingeniería de software: Un enfoque práctico*, 9na ed. Ciudad de México: McGraw-Hill, 2021.
- [2] B. Stroustrup, *The C++ Programming Language*, 4ta ed. Addison-Wesley Professional, 2013.
- [3] H. M. Deitel y P. J. Deitel, *Java: Cómo programar*, 10ma ed. Pearson Educación, 2016.
- [4] J. L. Hein, *Discrete Structures, Logic, and Computability*, 4ta ed. Jones & Bartlett Learning, 2017.
- [5] A. Silberschatz, H. F. Korth y S. Sudarshan, *Fundamentos de bases de datos*, 6ta ed. McGraw-Hill, 2014.

.2 Anexos

Link de GitHub:

<https://github.com/ginaq4457-glitch/APE05>